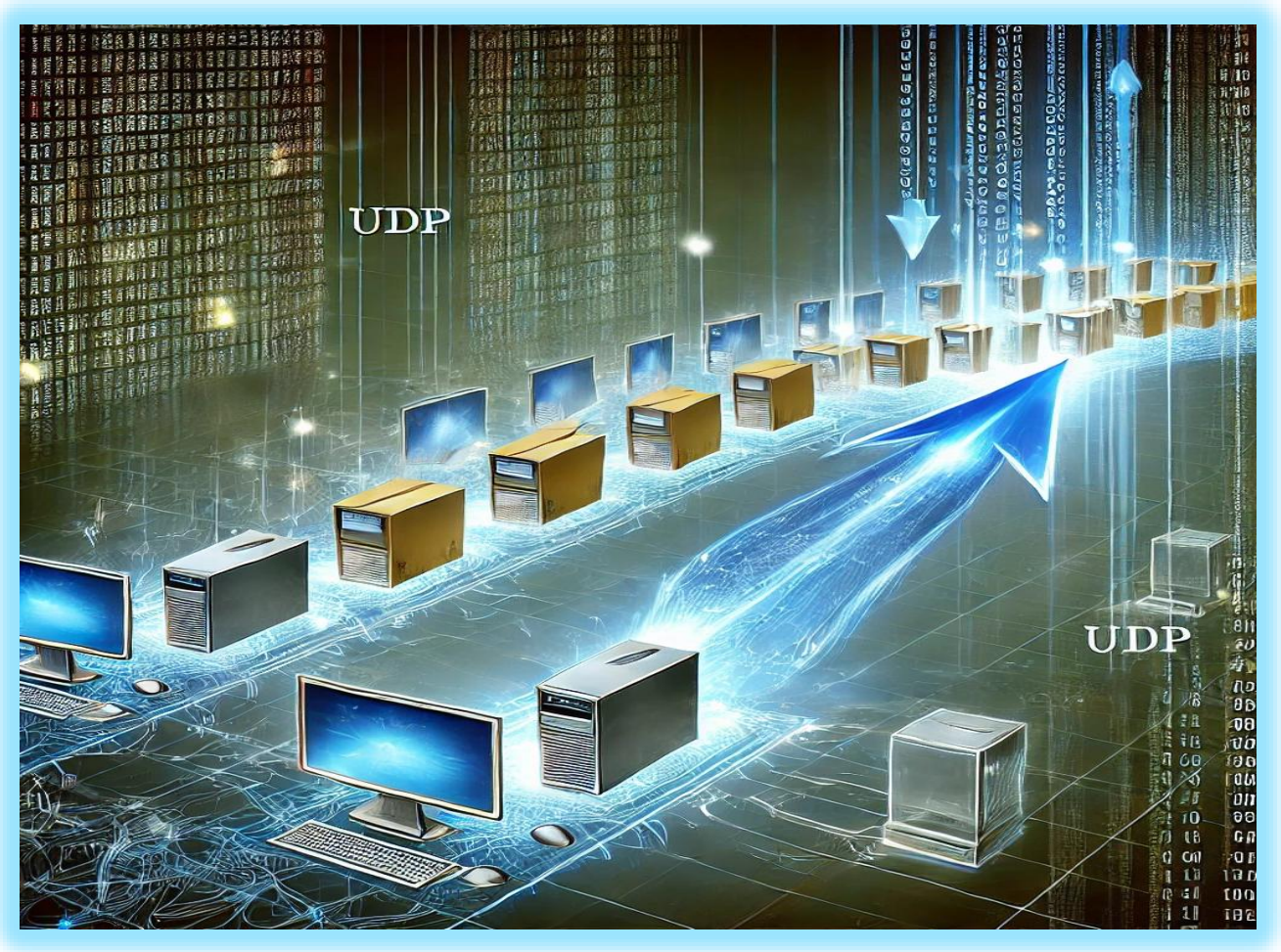
Roma, 12/05/2025

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Ingegneria Di Internet E Web 2023/2024

Progetto:

Trasferimento File Su UDP



A cura di:

Pompeo Alessio

Stoica Davide Edoardo

Caporaso Consuelo

Sommario

1. Obiettivo Del Progetto 5

2. Processi Client – Server 6

3. Meccanismo Di Comunicazione Client – Server 7

3.1. TOS – Type Of Service 8

4. Trasferimento UDP 10

5. Selective Repeat 12

6. Architettura Del Sistema 14

6.1. MakeFile 16

7. Specifiche Implementative 17

7.1. Instaurazione Della Connessione 17

7.2. Gestione Della Concorrenza 17

7.3. Multiplazione E Demultiplazione 18

7.4. Comando *list* – Visione File Disponibili 18

7.5. Comandi *get/put* – Dowload/Upload File 19

7.6. Trasferimento File 19

7.6.1. Principio Del Buffer Circolare 20

7.6.2. Three – Way – Handshake 21

7.6.3. Checksum 23

7.6.4. CodeBase64 24

7.6.5. Cifrario Di Cesare 24

7.6.6. Timeout E RTT 25

7.7. Comando *quit* 25

8. Esempi Di Funzionamento 26

8.1. Schermata Iniziale 26

8.2. Messaggi Di Comando 28

9. Limitazioni E Prospettive Di Miglioramento 34

9.1. Limitazioni Dovute all'Uso Di UDP 34

9.2. Restrizioni Sulla Dimensione Dei File E Sul Numero Massimo Di *Thread* 34

9.3. Criticità, Bugs E Futuri Miglioramenti 34

10. Manuale Di Installazione, Configurazione Ed Esecuzione 36

10.1. Esecuzione/Disinstallazione Del Codice Sul/Dal Proprio Dispositivo 36

10.2. Configurazione Dei Parametri (opzionale).....36

10.3. Support 37

11. Ambiente Di Sviluppo E Librerie 38

11.1. Piattaforma Di Sviluppo 38

11.2. Librerie Utilizzate 38

11.3. Specifiche Di Sistema E Sviluppo (Linux – Fedora) 38

11.3.1. Sistema Operativo E Kernel 38

11.3.2. Architettura CPU E Memoria 39

11.3.3. Versione Di GCC..... 40

11.3.4. Versione Di LIBC (glibc) 40

11.3.5. Versione Di MAKE 40

11.3.6. Versione Di CMAKE (se presente) 41

11.3.7. Versione Di GDB (Debugger) 41

11.4. Specifiche Di Sistema E Sviluppo (Linux – Ubuntu).....41

11.4.1. Sistema Operativo E Kernel 41

11.4.2. Architettura CPU E Memoria 41

11.4.3. Versione Di GCC..... 42

11.4.4. Versione Di LIBC (glibc) 42

11.4.5. Versione Di MAKE 42

11.4.6. Versione Di CMAKE (se presente) 43

11.4.7. Versione Di GDB (Debugger) 43

12. Valutazioni Prestazionali 44

12.1. Tabella 1 45

12.2. Tabella 2 46

12.3. Tabella 3 47

12.4. Tabella 4 48

12.5. Tabella 5 49

12.6. Tabella 6 50

12.7. Tabella 7 51

12.8. Tabella 8 52

12.9. Tabella 9 53

12.10. Tabella 10 54

12.11. Tabella 11 55

12.12. Tabella 12 56

12.13. Tabella 13 57

12.14. Tabella 14 58

12.15. Tabella 15 59

1. Obiettivo Del Progetto

Il progetto si propone di realizzare un'applicazione Client – Server che utilizzi il protocollo UDP per la trasmissione e il trasferimento dei file su rete in maniera affidabile; poiché UDP non garantisce affidabilità, l'applicazione dovrà implementare a livello applicativo un protocollo di trasferimento affidabile basato su:

- **Selective Repeat** con finestra di trasmissione di dimensione N → gestione indipendente di ACK e ritrasmissione per ogni pacchetto perso o non confermato;
- **Timeout Configurabile** → possibilità di un timeout fisso o adattivo, calcolato dinamicamente in base ai ritardi di rete osservati (RTT);
- **Simulazione di Perdita dei Pacchetti** → introduzione di una probabilità P di perdita dei pacchetti simulata localmente dal Mittente;
- **ACK/NAK e Ritrasmissione Selettiva** → in modo tale da garantire la consegna corretta ed in ordine dei segmenti.

2. Processi Client – Server

Il sistema realizzato è composto da due moduli, Client e Server, eseguiti su macchine differenti. In particolare:

- il **Server** si occupa di tutte le operazioni necessarie a fornire un servizio;
- il **Client**, dopo aver instaurato una connessione, invia al Server le richieste formulate dall'Utente.

Ogni processo invia e riceve messaggi attraverso un'interfaccia tra il livello applicativo e il livello di trasporto, un'interfaccia locale all'Host, chiamata *Socket*; il livello di trasporto nell'Host di destinazione, quindi, non trasferisce i dati direttamente ad un processo, ma ad una *Socket*, che funge da intermediario.

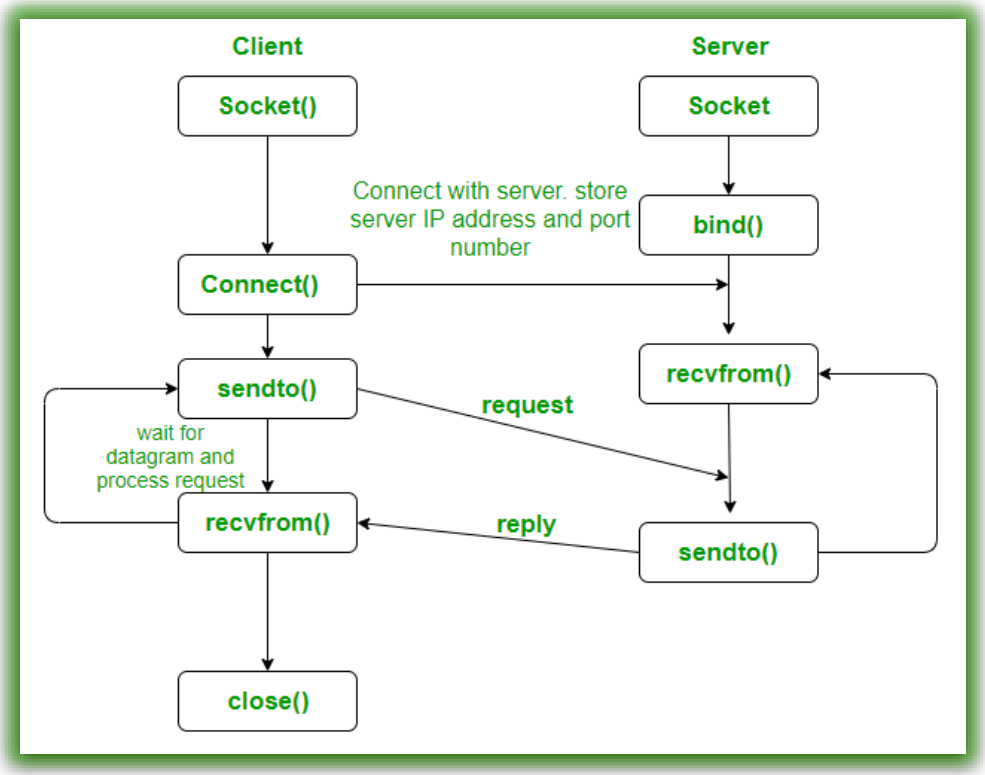


Figura 1: Connessione UDP

Nel nostro contesto, il ciclo di vita di una *Socket* è molto semplice:

- **apertura**;
- **lettura/scrittura**;
- **chiusura**;

il suo funzionamento è controllato dal Sistema Operativo.

Per interagire con il Sistema Operativo, abbiamo utilizzato le API – *Application Programming Interface*, delle *Socket di Berkeley*.

3. Meccanismo Di Comunicazione Client – Server

Il meccanismo di comunicazione tra Client e Server avviene come segue in figura

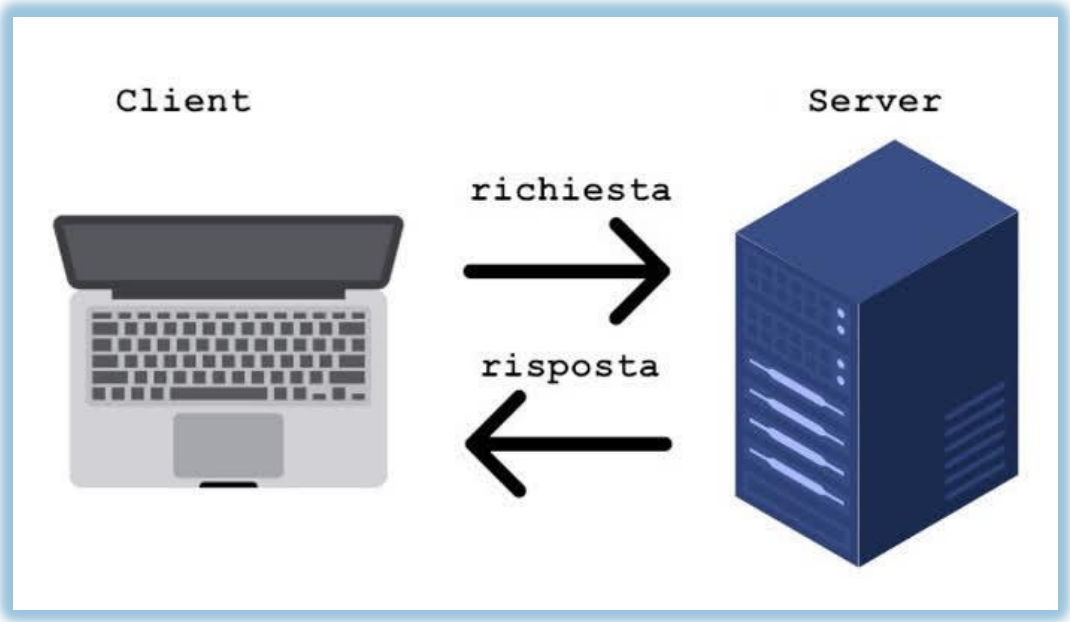


Figura 2: Meccanismo Di Comunicazione Client – Server

Affinché il Client ed il Server possano comunicare, è necessario instaurare una connessione (*Figura 1*).

Nel lato Server questo avviene attraverso sei fasi:

1. creazione e configurazione del Socket principale → il Server ciclicamente
   - crea un *Socket* UDP → *sock_main_thread*;
   - imposta le opzioni *SO_REUSEADDR* e *SO_REUSEPORT* per riutilizzare rispettivamente indirizzo e porta in caso di riavvio rapido del Server;
   - imposta il campo TOS – *Type Of Service* del pacchetto IP per dare priorità alta alla trasmissione;
   - prepara la struct *sockaddr_in* con indirizzo IP e porta → *server_port*, e fa il *bind()* del *Socket*;
2. attesa ricezione dati → il Server
   - si mette in ascolto, tramite la funzione *rcvfrom()*, su quel *Socket* principale;
   - imposta un timeout di 5 secondi per evitare di bloccarsi indefinitivamente;
   - riceve un messaggio iniziale dal Client contenente una richiesta di connessione ed una porta suggerita per le comunicazioni successive;
3. risposta e sincronizzazione → il Server
   - valida la porta ricevuta tramite la funzione *port_validation()*;
   - manda un pacchetto di sincronizzazione, tramite la funzione *sndMsg_sync()*, alla porta specificata dal Client per assicurarsi che quest'ultimo sia pronto → se la sincronizzazione fallisce, ritenta fino a 10 volte;
4. creazione del *Socket* dedicato per il Client → il Server
   - crea un nuovo *Socket* UDP → *sock*, legato ad una nuova porta specifica;
   - prepara la struct *server_addr* con indirizzo e porta e fa il *bind()* del *Socket*;
   - alloca memoria per *filepath*, che servirà a gestire il file che il Client vuole inviare o ricevere;

5. avvio *thread* per gestire il Client → viene creato un nuovo *thread*, tramite la funzione *pthread_create()*, che esegue la funzione *handle_request()* passando la struct *client_info*; tale *thread* gestirà tutta la comunicazione successiva con il Client in maniera indipendente;
6. logging e cleanup
   - viene registrato il tempo di connessione;
   - vengono aggiornati gli array globali per tracciare connessioni attive, quali
     - **connection_ports**;
     - **connection_socks**;
     - **connection_time**;
   - si chiude il Socket principale → *sock_main_thread* → il Server è pronto per accettare nuove richieste nel ciclo successivo.

3.1. TOS – *Type Of Service*

Il campo *TOS – Type Of Service* è un'opzione dell'intestazione IP che consente di specificare come un pacchetto deve essere trattato dai router lungo il percorso; non influisce sul contenuto, ma influenza il comportamento della rete, infatti, può modificare la qualità del servizio, la priorità, la velocità, l'affidabilità e la latenza del traffico; sui *Socket* IP viene impostato tramite l'opzione *setsockopt(sock, IPPROTO_IP, IP_TOS, &tos, sizeof(tos))*, dove:

- **sock:** descrittore del *Socket*;
- **IPPROTO_IP:** livello di protocollo → IP, nel nostro caso;
- **IP_TOS:** opzione da configurare;
- **&tos:** valore che vogliamo assegnare;
- **sizeof(tos):** dimensione del valore.

Nel lato Client, invece, ciò avviene in dieci fasi:

1. creazione del *Socket* UDP → tramite la funzione *socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)* si crea un *Socket* UDP, dove:
   - **AF_INET:** protocollo IPv4;
   - **SOCK_DGRAM:** datagramma UDP;
   - **IPPROTO_UDP:** protocollo UDP;
2. impostazione del TOS – *Type Of Service* → tramite l'opzione sopradescritta viene impostato il TOS per dare priorità alta ai pacchetti;
3. riutilizzo della porta e configurazione del timeout
   - permette di riutilizzare la porta se il *Socket* viene riavviato;
   - viene impostato il timeout di ricezione a 2 secondi per non bloccare indefinitivamente l'attesa della risposta;
4. *parsing* e validazione della porta del Server → viene convertita la porta in long e viene verificata che sia numerica;
5. preparazione dell'indirizzo del Server
   - viene compilato *si_other* con IP e porta del Server;
   - viene configurato allo stesso modo *servAddr* per l'utilizzo successivo in ricezione;
6. invio del primo messaggio → viene inviato un messaggio iniziale, contenente la porta, al Server → simulazione tentativo di *handshake* UDP;
7. attesa della risposta del Server → se il Server è attivo, risponde, altrimenti:
   - se scade il timeout di 2 secondi → errore controllato;

- se presente altro errore → viene stampato;

8. tentativi multipli per ricevere il messaggio di SYNC → il Client tenta fino a 10 volte di ricevere un messaggio di SYNC (messaggio di conferma da parte del Server) e ottiene la porta su cui il Server vuole stabilire la comunicazione effettiva;
9. comunicazione riuscita: *parsing* e stampa → il Client decodifica la porta ricevuta e conferma l'instaurazione della comunicazione con il Server;
10. chiusura del *Socket* iniziale → una volta ricevuta la porta corretta e confermata la sincronizzazione, il *Socket* iniziale viene chiuso;

dopo questa configurazione, il Client può comunicare con il Server attraverso i seguenti comandi:

- ***list*:** il Server invia al Client l'elenco dei file disponibili;
- ***get* <*filename*>:** il Client richiede il file desiderato al Server, a condizione che quest'ultimo lo possieda;
- ***put* <*filename*>:** il file desiderato viene prelevato dalla cartella Upload del Client e, successivamente, inviato al Server;
- ***modify* <*filename*>:** il file desiderato viene modificato attraverso l'editor di testo Vim;
- ***delete* <*filename*>:** il file desiderato viene eliminato dal Server;
- ***compress* <*filename*>:** il file desiderato viene compresso (riduzione delle dimensioni); il file risultante avrà un'estensione modificata;
- ***decompress* <*filename*>:** il file compresso desiderato viene decompresso, ripristinato al formato originale;
- ***quit*:** il Client termina la comunicazione con il Server.

4. Trasferimento UDP

UDP – *User Datagram Protocol* è un protocollo di trasporto che non prevede un meccanismo di *handshake* (processo di negoziazione tra due entità per stabilire, mantenere o terminare una comunicazione in modo affidabile) tra le unità di invio e di ricezione; di conseguenza, offre un servizio non orientato alla connessione.

Essendo un protocollo senza connessione, UDP non garantisce né il riordinamento dei pacchetti né la ritrasmissione di quelli persi, motivo per cui è considerato un protocollo non affidabile.

Nonostante la sua scarsa affidabilità, UDP viene utilizzato in diverse applicazioni per i seguenti motivi:

- maggiore controllo su quali dati inviare e quando inviarli;
- assenza di connessione → non essendoci *handshaking* tra le parti coinvolte, UDP non introduce ritardi nello stabilire una connessione e non conserva lo stato della comunicazione; il DNS – *Domain Name Systema* utilizza UDP proprio per questa ragione;
- dimensioni ridotte dell'intestazione → l'intestazione di UDP è di soli 8 byte, il che riduce l'overhead rispetto ad altri protocolli, come TCP.

UDP svolge solo le funzioni essenziali di un protocollo di trasporto, tra cui:

- **Multiplexing**, per l'indirizzamento dei dati;
- **Demultiplexing**, per la distribuzione dei dati ai processi corretti;
- **Checksum:** verifica gli errori.

In particolare, dopo aver ricevuto i messaggi dal processo applicativo, UDP aggiunge:

- il numero di porta di origine e il numero di porta di destinazione per consentire il Multiplexing ed il Demultiplexing;
- due ulteriori campi, tra cui il Checksum;

successivamente, il segmento UDP viene passato al livello di rete, che lo inserisce in un datagramma IP ed effettua un tentativo di consegna. Se la trasmissione ha successo, UDP utilizza il numero di porta di destinazione per recapitare i dati al processo applicativo corretto.

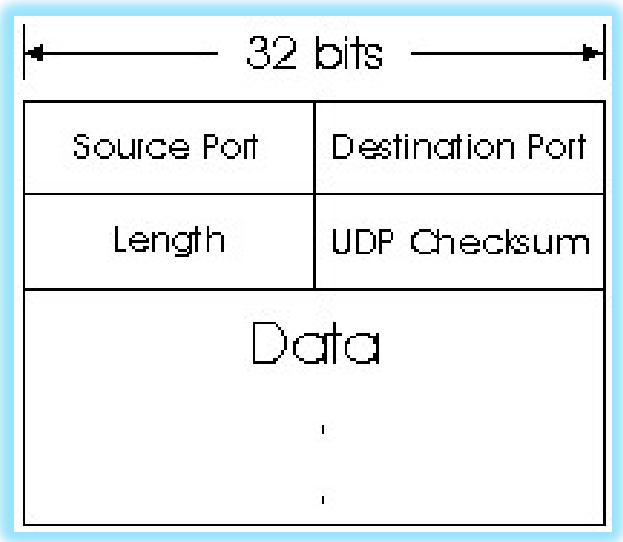


Figura 3a: Struttura Segmento UDP

Dalla *Figura 3a* possiamo osservare che l'intestazione (Header) di un pacchetto UDP è molto semplice e presenta solo quattro campi di 2 byte ciascuno:

1. **Source Port:** numero di porta della sorgente;
2. **Destination Port:** numero di porta della destinazione;
3. **Lenght:** lunghezza totale (numero di byte) del segmento UDP;
4. **Checksum:** verifica se i bit del segmento UDP sono stati alterati durante la trasmissione (utilizzato dal ricevente);
5. **Data:** contenuto effettivo da inviare.

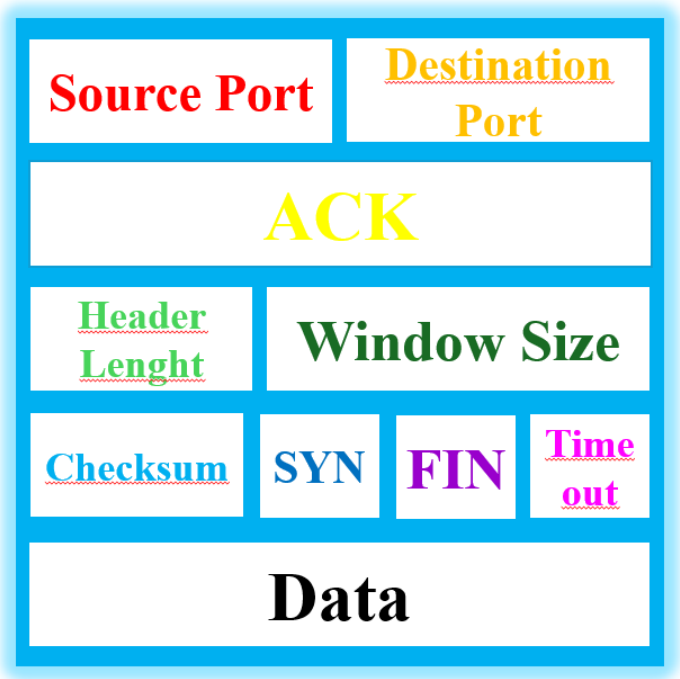


Figura 3b: Struttura Segmento UDP – made by us

Dalla *Figura* 3b, invece, possiamo osservare la struttura di un segmento UDP *made by us*, che presenta nove campi, sempre per un totale di 32 bit:

1. **Source Port:** numero di porta della sorgente;
2. **Destination Port:** numero di porta della destinazione;
3. **ACK – Acknowledgment:** messaggio di conferma per indicare che un dato pacchetto o messaggio è stato ricevuto correttamente;
4. **Header Lenght:** lunghezza totale (numero di byte) del segmento UDP;
5. **Window Size (o Finestra di Ricezione):** indica quanti byte (o pacchetti) un ricevente può accettare prima di dover inviare un ACK → controlla quanta roba può essere spedita senza aspettare conferma;
6. **Checksum:** verifica se i bit del segmento UDP sono stati alterati durante la trasmissione (utilizzato dal ricevente);
7. **SYN – Synchronize:** flag TCP utilizzato per iniziare una connessione tra due host → sincronizza i numeri di sequenza iniziali tra mittente e destinatario;
   FIN – Finish: flag TCP utilizzato per chiudere una connessione tra due host in modo ordinato → indica che non verranno più inviati dati da quella parte;
8. **Timeout:** intervallo di tempo massimo concesso al sistema per il completamento di un'operazione o per ricevere una risposta, dopo il quale si considera l'operazione fallita o scaduta ed il sistema ritenta, segnala l'errore o termina l'operazione → limite di attesa, se entro quel tempo non succede qualcosa, il sistema smette di aspettare;
9. **Data:** contenuto effettivo da inviare.

5. Selective Repeat

Per garantire l'affidabilità della trasmissione dei dati su un protocollo UDP, è stato necessario implementare a livello applicativo il protocollo *Selective Repeat*; si tratta di un meccanismo di controllo che permette di gestire in modo efficiente la ritrasmissione dei pacchetti → ogni pacchetto trasmesso viene identificato da un numero di sequenza e, una volta ricevuto, il destinatario invia un messaggio di riconoscimento (ACK) per confermarne la corretta ricezione.

A differenza del protocollo Go – Back – N, in cui la perdita di un singolo pacchetto comporta la ritrasmissione di tutti i pacchetti successivi già inviati nella pipeline, il *Selective Repeat* consente di ritrasmettere esclusivamente i pacchetti effettivamente persi o corrotti; in questo modo si evitano ritrasmissioni inutili, riducendo il traffico di rete e migliorando l'efficienza della comunicazione.

Dal punto di vista implementativo, sia il mittente che il destinatario dispongono di un buffer in cui vengono temporaneamente memorizzati i pacchetti, ciascuno associato ad un numero di sequenza; il mittente mantiene una finestra di trasmissione che gli consente di inviare più pacchetti prima di ricevere un riscontro, mentre il destinatario conserva i pacchetti ricevuti, anche se fuori ordine, e li riassembla una volta che tutti i pacchetti della sequenza sono disponibili; così facendo si migliora notevolmente l'efficienza della comunicazione, riducendo la latenza e permettendo un recupero più rapido dei dati in caso di perdita; inoltre, questo approccio garantisce una maggiore affidabilità nelle reti con alta latenza o con una percentuale elevata di errori, evitando congestionamenti inutili e assicurando un trasferimento più fluido e stabile.

Di seguito viene riportato il comportamento del Mittente e del Destinatario:

Mittente:

- utilizza una finestra di trasmissione che può contenere fino a N pacchetti provenienti dall'applicazione, che possono essere inviati in parallelo;
- ogni pacchetto ha un timer associato, utile per la ritrasmissione nel caso in cui non venga ricevuto alcun ACK entro il tempo stabilito;
- quando riceve un ACK per un pacchetto, questo viene classificato come "*Ricevuto*";
- se l'ACK ricevuto corrisponde al primo pacchetto nella finestra, l'indice della finestra si sposta sul pacchetto con il numero di sequenza più basso non ancora confermato;
- questo scorrimento della finestra permette di caricare e inviare nuovi pacchetti.

Destinatario:

- se riceve un pacchetto all'interno della sua finestra di ricezione, invia un ACK selettivo;
- se il pacchetto non era già stato ricevuto, viene inserito nel buffer;
- se il numero di sequenza del pacchetto ricevuto coincide con la base della finestra di ricezione, il pacchetto viene consegnato al livello superiore insieme a tutti gli altri pacchetti ricevuti con numeri di sequenza consecutivi presenti nel buffer;
- in questo modo si garantisce il corretto ordinamento nella consegna dei pacchetti al processo applicativo.

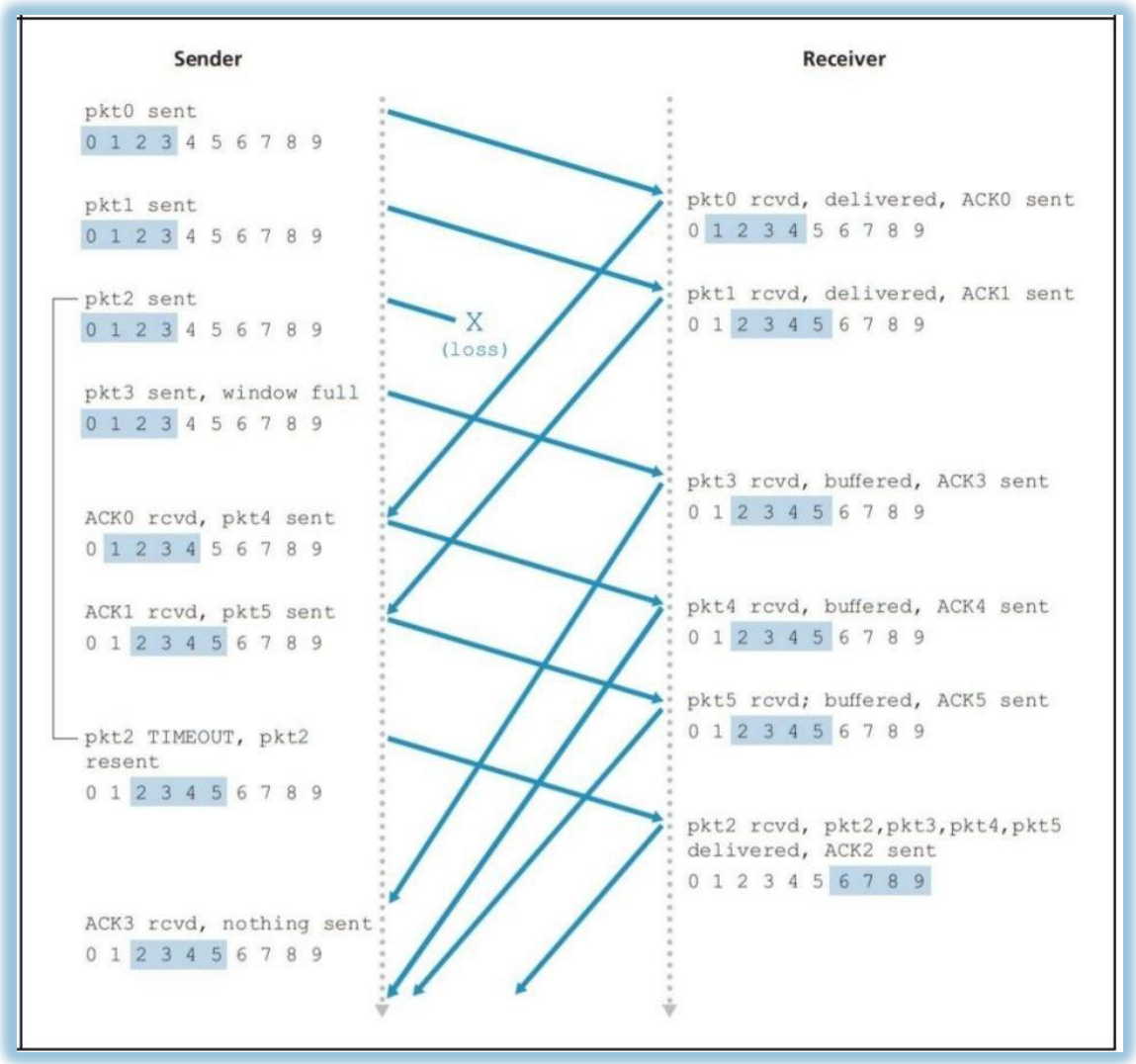


Figura 4a: Protocollo Selective Repeat

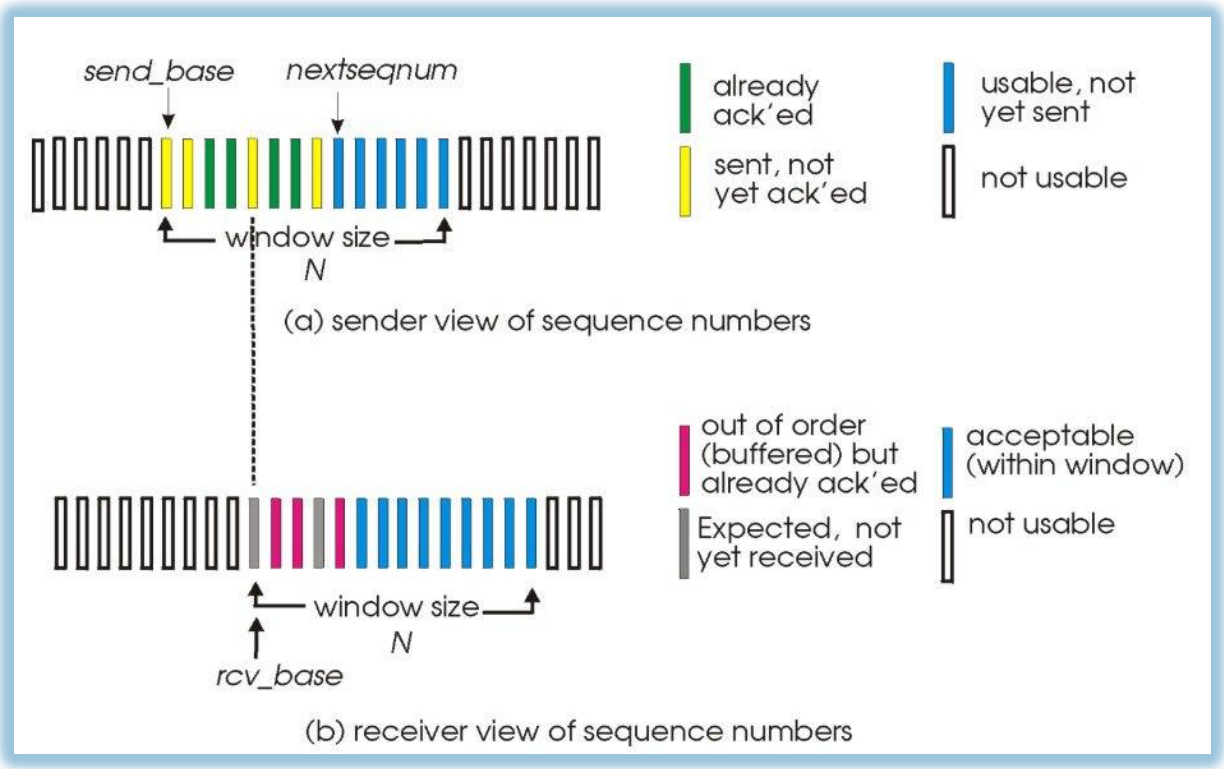


Figura 4b: Protocollo Selective Repeat

6. Architettura Del Sistema

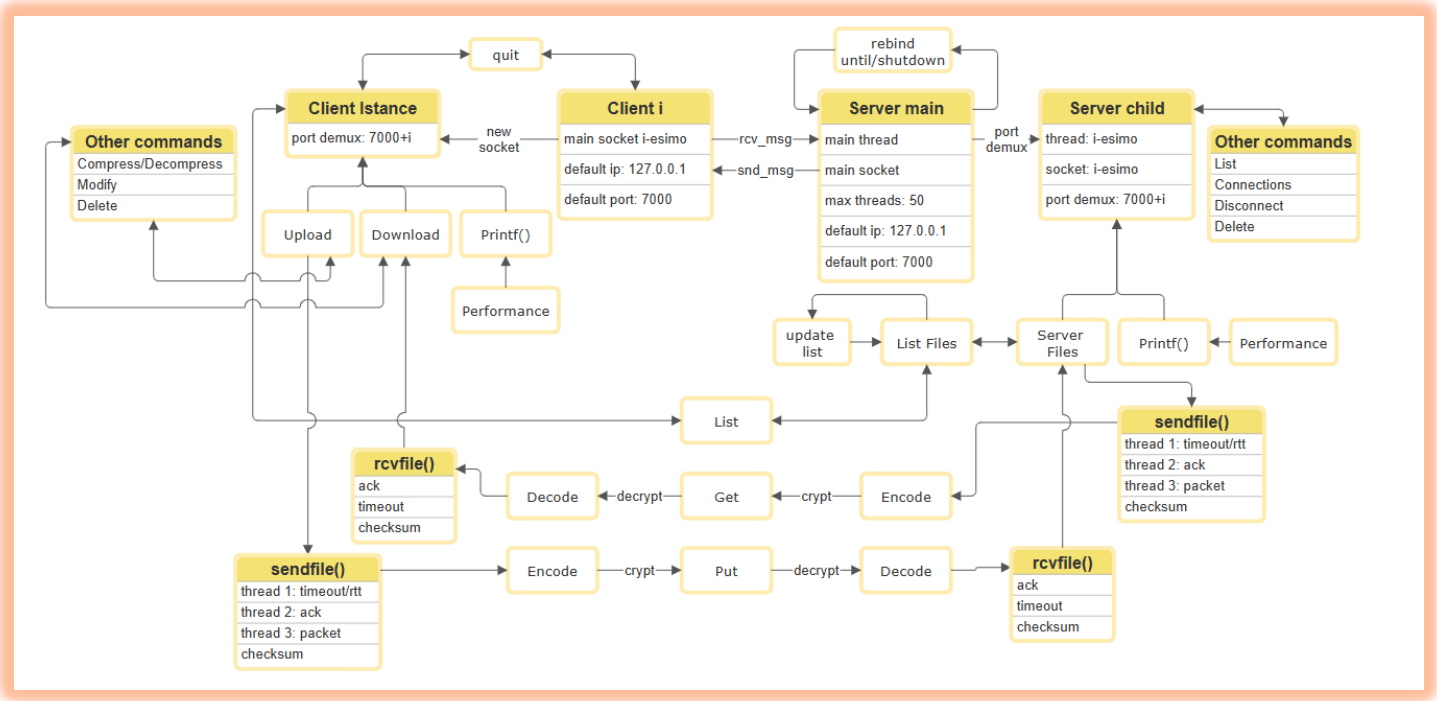


Figura 5a: Grafo Architettura Sistema

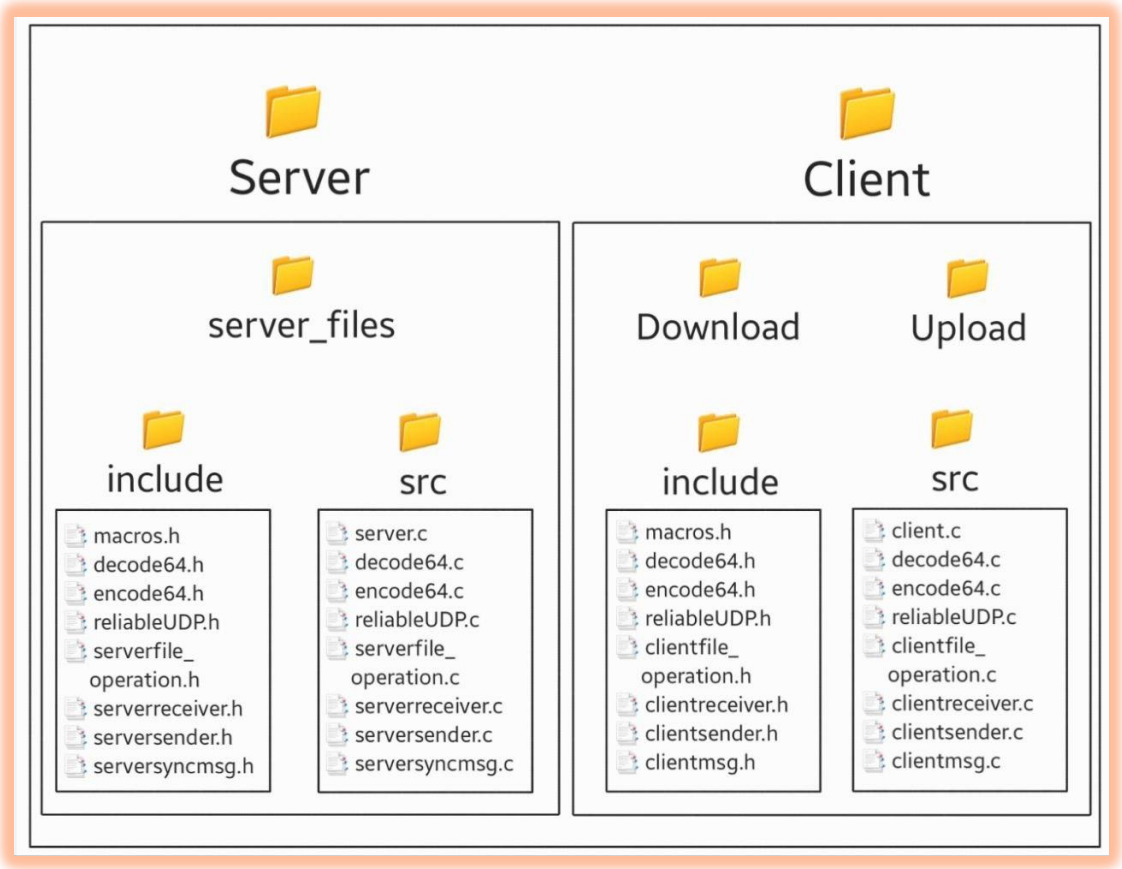


Figura 5b: Architettura Sistema

Lato Server:

- **server_ files:** cartella contenente tutti i file posseduti dal Server;
- **src:** cartella contenente l'implementazione delle funzionalità del Server →
  - **decode64.c:** decodifica il file in ingresso attraverso il codebase64;

    - **encode64.c:** codifica il file in ingresso attraverso il codebase64;
    - **reliableUDP.c:** file contenente l'implementazione del *checksum*;
    - **Server.c:** gestione delle richieste da parte del Client (*list*, *get*, *put*, ecc…)
    - **serverfileoperation.c:** gestisce la lavorazione di ciascun file (apertura, chiusura, creazione, ecc…);
    - **serverreceiver.c:** gestisce la ricezione dei file provenienti dal Client;
    - **serversender.c:** gestisce l'invio dei file verso il Client;
    - **serversyncmsg.c:** messaggio di sincronizzazione all'inizio della connessione;
- **include:** cartella contenente gli header file – prototipo delle funzioni →
    - **decode64.h**
    - **encode64.h**
    - **macros.h**
    - **reliableUDP.h**
    - **serverfileoperation.h**
    - **serverreceiver.h**
    - **serversender.h**
    - **serversyncmsg.h**
- **files_list:** file di testo contenente la lista dei nomi dei file posseduti dal Server;
- **makefile**
- **README.md:** manuale istruzioni;
- **run_server.sh:** script di avvio del Server;

Lato Client:

- **Download:** contiene i file ricevuti dal Server tramite la richiesta *get filename.txt*;
- **Upload:** contiene i file che possono essere inviati al Server tramite il comando *put filename.txt*;
- **src:** cartella contenente l'implementazione delle funzionalità del Client →
    - **decode64.c:** decodifica il file in ingresso attraverso il codebase64;
    - **encode64.c**: codifica il file in ingresso attraverso il codebase64;
    - **reliableUDP.c:** file contenente l'implementazione del *checksum*;
    - **Client.c:** gestione delle richieste da parte dell'Utente (*list*, *get*, *put*, ecc…);
    - **clientfileoperation.c:** gestisce la lavorazione di ciascun file (apertura, chiusura, creazione, ecc…);
    - **ClientReceiver.c:** gestisce la ricezione dei file provenienti dal Server;
    - **ClientSender.c:** gestisce l'invio dei file verso il Server;
    - **clientsyncmsg.c:** messaggio di sincronizzazione all'inizio della connessione;
- **include:** cartella contenente gli header file – prototipo delle funzioni →
    - **decode64.h**
    - **encode64.h**
    - **macros.h**
    - **reliableUDP.h**
    - **clientfileoperation.h**
    - **ClientReceiver.h**
    - **ClientSender.h**
    - **clientsyncmsg.h**
- **makefile**
- **run_client.sh:** script di avvio del Client.

6.1. MakeFile

Il *Makefile* è un file di automazione della compilazione che permette di gestire in maniera efficiente la costruzione dell'intero progetto, orchestrando la compilazione dei file sorgente ed il collegamento degli oggetti per ottenere l'eseguibile finale; in questo progetto, il suo utilizzo semplifica significativamente il processo di build, consentendo di ricompilare solo le parti modificate, riducendo il rischio di errori manuali e garantendo una gestione più rapida ed efficiente delle dipendenze; nel *Makefile* fornito vengono definite alcune variabili chiave:

- **CC** e **CFLAGS:** indicano il compilatore scelto (*gcc*) e le opzioni di compilazione, come:
  - ottimizzazioni → *–O2*;
  - supporto per i *thread* → *–pthread*;
  - numerosi flag che abilitano controlli sui warning e sul rispetto degli standard di codifica → *–Wall*, *–Wextra*, *–pedantic*, ecc…, assicurando una maggiore sicurezza e qualità del codice;
- **LDFLAGS:** specifica le opzioni per il linker, includendo le librerie esterne necessarie (ad esempio, *zlib*) ed il supporto per i *thread*;
- **SRC_DIR**, **INCLUDE_DIR** e **SRC_FILES:** definiscono la struttura del progetto, indicando rispettivamente la cartella dei file sorgente, quella degli header e l'elenco di tutti i file .c da compilare;
- **OBJ_FILES** e **TARGET:** vengono utilizzati per costruire la lista dei file oggetto compilati a partire dai file sorgente, e per definire il nome dell'eseguibile finale → *my_client*;

le regole del *Makefile* definiscono come trasformare i file sorgente in oggetti e come collegare questi ultimi per ottenere l'eseguibile; in particolare, grazie alla regola *clean*, è possibile eliminare i file oggetto e l'eseguibile, facilitando così la pulizia del progetto.

7. Specifiche Implementative

7.1. Instaurazione Della Connessione

Quando il Server, in ascolto su una determinata porta, riceve una richiesta da parte del Client, viene instaurata una connessione tra i due; la porta di ascolto del Server e la porta di connessione del Client devono coincidere affinché la connessione abbia luogo.

Ogni volta che il Client desidera collegarsi al nostro Server, utilizza una porta predefinita o di default.

Il Server progettato può accettare richieste da parte di più Client contemporaneamente e gestire, quindi, porte differenti per ciascuno di essi; a tal proposito, il Server assegnerà sequenzialmente, partendo dalla porta di default, una porta specifica a ciascun Client richiedente; il Client, una volta ricevuta la porta, stabilisce una nuova connessione con il Server, che ascolterà su quella porta.

Per garantire l'affidabilità nella gestione delle diverse porte, quando il Client riceve correttamente la porta assegnatagli dal Server, invierà a quest'ultimo un messaggio di conferma (ACK), così da confermare la corretta ricezione; garantita l'affidabilità, il Server è pronto a ricevere i messaggi.

In termini di implementazione, sia il Server che il Client devono creare un canale di comunicazione utilizzando la funzione *Socket*; per poterla utilizzare, sono stati usati parametri specifici per il protocollo UDP:

- **SOCK DGRAM:** supporta i datagrammi – messaggi non affidabili;
- **IPPROTO UDP:** campo formale specifico per il protocollo UDP;

inoltre, dovranno definire una struttura in cui specificare la porta che verrà utilizzata ed un campo predefinito AF_INET; quest'ultimo, assieme al parametro SOCK DGRAM, consentirà di stabilire una trasmissione UDP.

A differenza del Client, il Server eseguirà l'operazione di *bind()* per informare il sistema operativo su quale processo dovrà ricevere i dati provenienti dalla rete; il Server si metterà poi in attesa di ricevere dati.

7.2. Gestione Della Concorrenza

Per far sì che il Server riesca a gestire più Client contemporaneamente, abbiamo deciso di strutturarlo utilizzando il principio del *multithreading*, anziché il multiprocesso, poiché, nei Server di rete con carichi di lavoro elevati, esiste un limite alla multiprogrammazione che, se superato, comporta il rifiuto da parte del Server di ulteriori richieste da parte dei Client.

Dal punto di vista implementativo, quindi, avremo un *thread* Server per ogni Client che si connette.

Inoltre, in termini di risorse, l'utilizzo dei *thread* è meno impegnativo, di fatti, un'applicazione *multithread* implica la creazione di un unico processo con conseguente divisione dello stack tra i vari *thread*, al contrario, l'approccio multiprocesso richiede una maggiore allocazione di memoria ed un maggiore utilizzo delle risorse della macchina.

Quindi, per gestire le risorse in modo ottimale e sicuro, ogni *thread* avrà il proprio stack privato, allocato all'interno dello stack fisico del processo principale → ogni Client avrà una struttura dedicata sul Server, con una *Socket* e una porta privata → controllo efficiente della concorrenza → nessun utilizzo di risorse di sincronizzazione (semafori, mutex, ecc…) → risorse non condivise.

Dal punto di vista delle prestazioni, invece, l'utilizzo dei *thread* comporta diversi vantaggi:

- maggiore scalabilità → consente la gestione concorrente di un elevato numero di Client (50 *Threads*);
- maggiore resilienza → eventuali problemi relativi ad un singolo Client rimangono isolati, evitando impatti sull'intero sistema;
- supporto alle operazioni parallele → consente l'esecuzione simultanea di comandi ed il trasferimento parallelo di più file;
- riduzione dell'overhead di sistema → comporta un minor carico sulla rete ed una diminuzione della latenza;
- efficiente sincronizzazione dei processi, mediante l'uso di semafori e mutex.

7.3. Multiplazione e Demultiplazione

La **Multiplazione** e la **Demultiplazione** delle porte consentono una gestione dinamica e parallela delle connessioni tra Client e Server.

Multiplazione delle Porte → quando il Server viene avviato, viene configurato con un indirizzo IP ed una porta predefiniti (ad esempio, indirizzo IP: 127.0.0.1, porta: 7777); il *thread* principale del Server si occupa di gestire tutte le richieste in ingresso su questa porta di default; i Client si collegano all'indirizzo IP e alla porta del Server prima definiti, utilizzandoli come punto di ingresso unificato per la comunicazione.

Demultiplazione delle Porte → stabilita la connessione iniziale, il Server esegue un controllo ciclico, tramite la funzione *Port Validation*, per individuare la prima porta disponibile e reindirizzare ciascun Client verso un *thread* secondario dedicato, associato ad una porta diversa (ad esempio, 7778, 7779, ecc…); in questo modo, il *thread* principale rimane libero per accettare nuove connessioni, evitando che una singola richiesta possa bloccare o rallentare l'intero sistema.

La Demultiplazione delle Porte abilita una comunicazione *multithreading*, migliorando l'efficienza complessiva; inoltre, la suddivisione delle connessioni su porte differenti aumenta la scalabilità del servizio → ogni *thread child* può gestire in maniera indipendente il proprio flusso di dati; in ambienti di rete complessi, questo approccio consente una gestione più robusta delle sessioni, riducendo il rischio di congestionamento e garantendo un'esperienza più stabile e fluida per tutti i Client connessi.

7.4. Comando *list* – Visione File Disponibili

Uno tra i servizi che il Client può richiedere al Server, una volta stabilita la connessione, è il comando *list*; tale comando restituisce al Client una lista dei file disponibili appartenenti al Server; in questo modo, il Client potrà visualizzare il nome del file che desidera e, in seguito, richiederlo tramite il comando *get* <*filename*>.

Lato Client → dopo aver inviato, tramite *sendto()*, il messaggio *list* al Server, il Client riceverà una duplice risposta da parte del Server:

1. numero di file disponibili nel Server;
2. uno per volta, tutti i nomi dei file disponibili nel Server;

una volta ottenuti entrambi i messaggi di risposta, il Client farà visionare all'Utente la lista dei file contenuti nel Server.

Lato Server → dopo aver ricevuto, tramite *rcvfrom()*, il messaggio *list* da parte del Client, il Server:

- ricostruisce il path – assoluto del file *files_list.txt*, in cui sono presenti i nomi dei file disponibili → abbiamo realizzato la funzione *obtain_path()*, che restituisce, a seconda del comando effettuato dal Client, il path dell'istruzione richiesta;
- partiziona il contenuto del file *files_list.txt* tramite la funzione *read file list()*, che fornisce, simultaneamente, l'esatto numero di file presenti nel Server ed un buffer di stringhe in cui ogni singolo elemento è il nome di un file;
- invia il numero di file disponibili;
- invia, uno per volta, i nomi dei file disponibili.

7.5. Comandi *get/put* – Dowload/Upload File

Due ulteriori servizi che il Client può richiedere, e offrire, al Server, una volta stabilita la connessione, sono i comandi *get* e *put*:

- ***get* <*filename*>:** consente al Client di scaricare un file desiderato tramite il Server → condizione necessaria e sufficiente è che il file sia presente all'interno della cartella *server_files* e all'interno del file *files_list.txt*, entrambi del Server;
- ***put* <*filename*>:** consente al Client di caricare un file nel Server, consentendo a quest'ultimo di memorizzare il file nella lista dei file disponibili, aggiornando il file *files_list.txt*; in questo modo se un altro Client dovesse richiedere lo stesso file in futuro, il Server potrebbe facilmente consentirne il download → condizione necessaria e sufficiente è che il file sia presente nella cartella Upload del Client.

7.6. Trasferimento File

Nel livello applicativo del lato Mittente, per realizzare un trasferimento di dati affidabile senza compromettere efficienza e velocità, la nostra scelta implementativa è stata l'utilizzo di tre *thread* operati in concorrenza (*multithreading*); ad ogni *thread* è stata assegnata una funzionalità/responsabilità differente, in modo tale da suddividere il lavoro in maniera proporzionale; i tre *thread* in questione sono stati suddivisi nel seguente modo:

1. **Thread_1:** gestore dei pacchetti → legge i dati dal file, li inserisce nel pacchetto con numero di sequenza associato e li invia;
2. **Thread_2:** gestore del messaggio di acknowledgement (ACK);
3. **Thread_3 – Main Thread:** gestore del timeout → gestisce le ritrasmissioni dei pacchetti persi;

per gestire l'invio dei pacchetti, abbiamo deciso di instaurare un'unica struct privata con all'interno un array di strutture associate alla gestione di un singolo pacchetto, in questo modo, tutti i *thread*, lavorando in contemporanea, sono a conoscenza della possibile perdita, ritrasmissione o ricezione di un dato pacchetto (identificato dal suo Seqnum); ciascun pacchetto scambiato tra Server e Client è, quindi, rappresentato come una *struct_and_pack*, i

cui cinque campi sono:

1. **Data:** contiene l'informazione da trasportare → buffer;
2. **Acked:** permette di capire se il pacchetto è stato riscontrato o meno → booleano;
3. **Seqnum:** numero di sequenza compreso tra 0 ed N → intero;
4. **Finished:** indica l'ultimo pacchetto della scomposizione del file da inviare → booleano;
5. **Retransmitted:** permette di capire se il pacchetto è stato ritrasmesso → booleano.

Nel livello applicativo del lato Destinatario, invece, la nostra scelta implementativa è stata l'utilizzo di un approccio mono – processo, costituito da un unico flusso d'esecuzione responsabile di operazioni di ricezione, invio dell'ACK e scrittura su un file; ciascun pacchetto scambiato tra Server e Client è, quindi, rappresentato come una *struct_and_pack*, i cui quattro campi sono:

1. **Data:** contiene l'informazione da trasportare → buffer;
2. **Acked:** permette di capire se il pacchetto è stato riscontrato o meno → booleano;
3. **Seqnum:** numero di sequenza compreso tra 0 ed N → intero;
4. **Finished:** indica l'ultimo pacchetto della scomposizione del file da inviare → booleano.

7.6.1. Principio del Buffer Circolare

Per implementare la ripetizione selettiva abbiamo deciso di utilizzare un *Buffer Circolare* di dimensione N ed alcune variabili, denotate con W, W_Sender ed W_Receiver, per tenere traccia della posizione delle finestre.

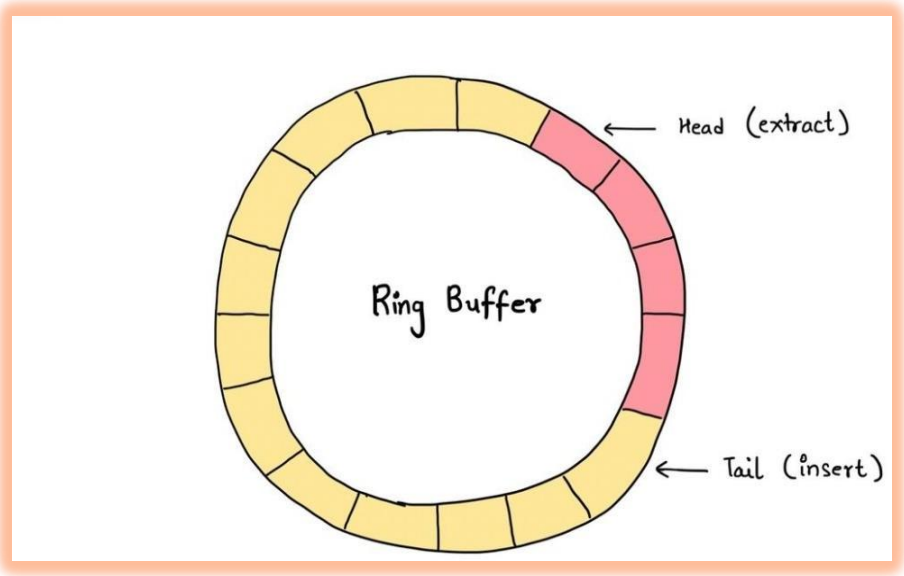


Figura 6: Buffer Circolare

Ogni pacchetto è associato ad un numero di sequenza (Seqnum), che indica la sua posizione nel buffer prima di essere inviato e dopo essere stato ricevuto.

Lo scorrimento della finestra viene gestito incrementando le variabili associate al buffer circolare.

Nel lato Mittente, i *thread* utilizzano il buffer circolare per la bufferizzazione dei pacchetti inviati; se un pacchetto o l'ACK ad esso relativo viene perso, il *thread* responsabile lo ritrasmette prelevandolo dal buffer; il **Thread_2** libera il buffer una volta che il pacchetto non è più necessario.

Nel lato Destinatario, la bufferizzazione dei pacchetti viene effettuata poiché i pacchetti potrebbero non arrivare nell'ordine corretto.

Per gestire correttamente la terminazione della trasmissione viene applicata una *final_job* → quando il **Thread_1** completa il suo compito, notifica il **Thread_2**, il quale invia un pacchetto con un numero di sequenza (Seqnum) al di fuori della dimensione dei buffer di invio e ricezione, segnalando così la fine della trasmissione; il destinatario riconosce il pacchetto con numero di sequenza maggiore o uguale ad N e comprende che ha ricevuto l'intero file; il **Thread_2** termina il suo compito al ricevimento dell'ultimo ACK, che viene inviato dal Destinatario una o tre volte, a seconda se sia il Client o il Server.

Ad ogni pacchetto viene associato un timer, che scade in caso di perdita del pacchetto in rete o mancato ricevimento dell'ACK associato al pacchetto in questione → Timeout; per gestire quest'ultimo abbiamo utilizzato una *Linked List*, gestita dal **Thread_3**; ogni elemento, o nodo, della lista è una struct contenente i seguenti quattro campi:

1. **Seqnum:** numero di sequenza compreso tra 0 ed N → intero;
2. **Finished:** indica l'ultimo pacchetto della scomposizione del file da inviare → booleano;
3. **Timeval** → struct;
4. puntatore al nodo successivo.

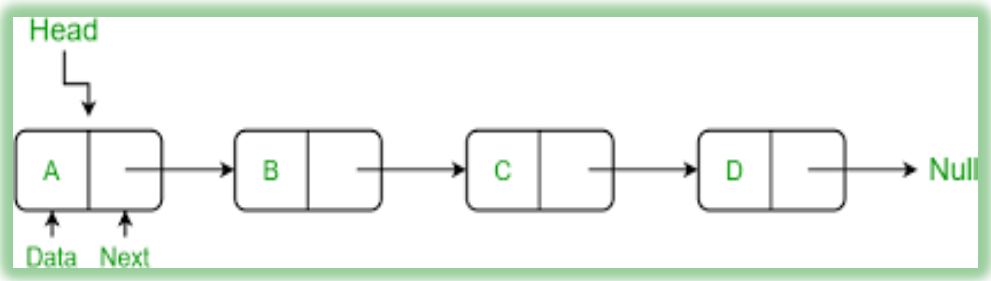


Figura 7: Linked List

Quando un pacchetto viene inviato, viene aggiunto alla fine di una Linked List; il **Thread_3** controlla costantemente il tempo di volo del primo elemento della lista, il pacchetto inviato da più tempo, e lo confronta con il valore del timeout, specificato come variabile globale; se il **Thread_2** riceve un ACK per un pacchetto di cui non ha ancora ricevuto conferma, scorre la Linked List per individuare il nodo corrispondente, gli azzera il timer e lo elimina.

7.6.2. Three – Way – Handshake

L'affidabilità deve essere garantita non solo nel trasferimento dei file, ma anche nello scambio di messaggi/informazioni tra Client e Server → affidabilità nell'invio di comandi semplici; la strategia che abbiamo deciso di adottare è nota come *Three – Way – Handshake*, utilizzata nel protocollo TCP per stabilire una connessione; i messaggi vengono inviati e ricevuti tramite le funzioni *sndMsg()* e *rcvMsg()*, che a loro volta utilizzano le funzioni *sendto()* e *rcvfrom()*.

Il processo di connessione inizia con l'invio del pacchetto INFO da parte del Mittente e termina con l'invio dell'ACK da parte del Destinatario (ricezione dell'ACK da parte del Mittente); per essere sicuri che il Mittente abbia ricevuto correttamente l'ACK (il Destinatario abbia mandato correttamente l'ACK), abbiamo impostato un timeout sulle *rcvfrom()* di entrambi.

Se il Mittente non riceve l'ACK da parte del Destinatario, il timer della *rcvfrom()* scade ed il

Mittente tenta per dieci volte di ritrasmettere il pacchetto INFO (non tre volte come da definizione), fino a quando non riceve l'ACK o un messaggio di errore; d'altra parte, ogni volta che il Destinatario riceve un messaggio, invia un ACK ed imposta il timeout della *rcvfrom()*, in modo tale che il Mittente possa ritrasmettere più volte se necessario, se scade il timeout, il Destinatario termina con un messaggio di errore.

All'inizio del processo, il Mittente invia un messaggio di tipo SYN per avviare la connessione, il Destinatario risponde con un messaggio di tipo SYNACK per confermare la ricezione del messaggio di tipo SYN ed indicare che è pronto alla comunicazione ed il Mittente risponde con un ACK per completare il processo di *Handshake* e stabilire la connessione; se il Mittente non riceve un messaggio di tipo SYNACK, ritenta l'invio di un messaggio di tipo SYN fino a dieci volte e se, dopo questi tentativi, non riceve risposta, considera la connessione fallita; il Destinatario continua ad inviare messaggi di tipo SYNACK ogni volta che riceve un messaggio di tipo SYN; se il Destinatario smette di ricevere messaggi di tipo SYN ed il timeout scade, verifica se ha ricevuto almeno un messaggio di tipo SYN valido; se il Destinatario ha ricevuto almeno un messaggio di tipo SYN, considera la connessione stabilita dopo aver ricevuto l'ACK dal Mittente.

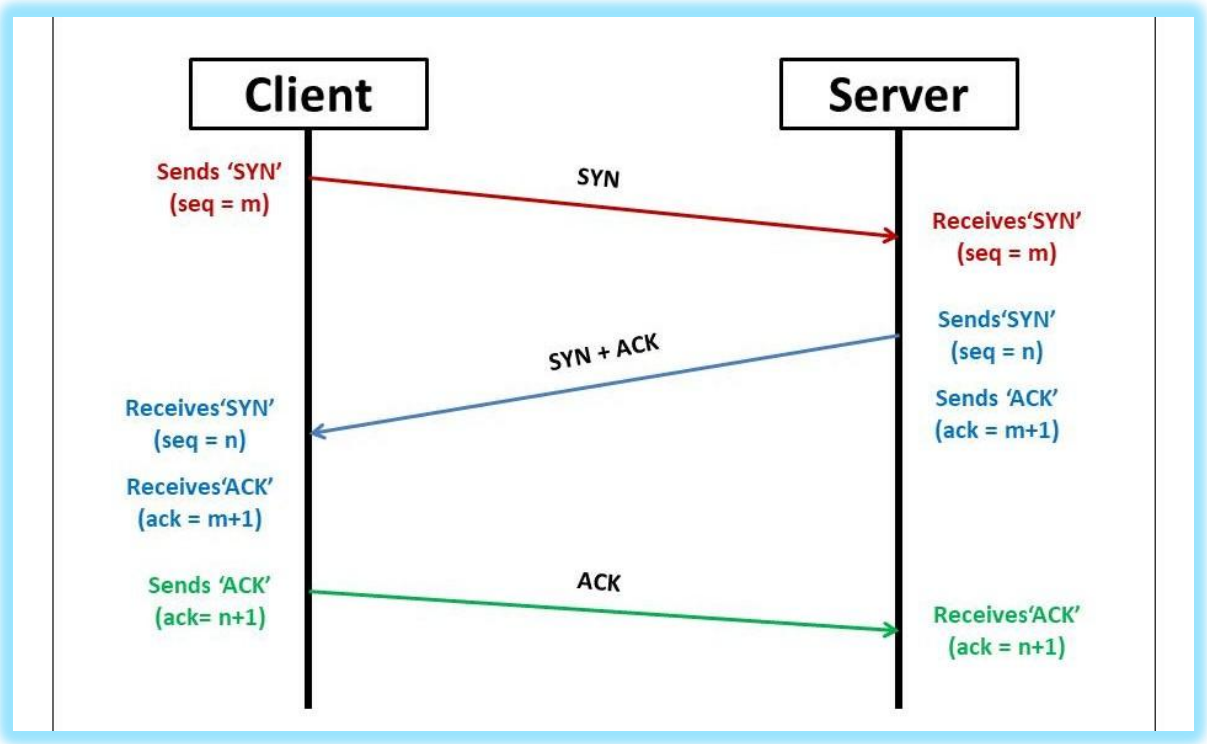


Figura 8a: Scambio Di Messaggi

Alla fine del processo, se tutto è andato a buon fine, il Mittente invia un messaggio di tipo FIN al Destinatario per segnalare la fine della comunicazione ed il Destinatario risponde con un messaggio di tipo FINACK per confermare la ricezione di un messaggio di tipo FIN; se il Mittente riceve un messaggio di tipo FINACK, considera la chiusura della comunicazione avvenuta con successo e smette di inviare messaggi di tipo FIN; se il Mittente non riceve un messaggio di tipo FINACK, ritenta l'invio di un messaggio di tipo FIN fino a dieci volte e se, dopo i tentativi, riceve un messaggio di tipo FINACK, conclude lo scambio correttamente; il Destinatario continua ad inviare un messaggio di tipo FINACK ogni volta che riceve un messaggio di tipo FIN; se il Destinatario smette di ricevere un messaggio di tipo FIN ed il timeout scade, verifica se ha ricevuto almeno un messaggio di tipo FIN; se il Destinatario ha ricevuto almeno un messaggio di tipo FIN, considera lo scambio completato con successo e

restituisce il messaggio INFO inizialmente ricevuto.

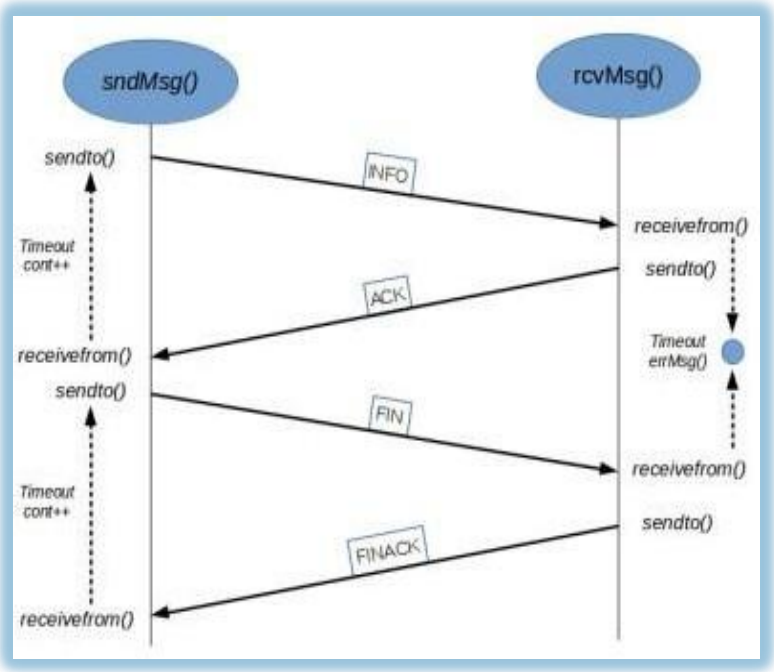


Figura 8b: Scambio Di Messaggi

7.6.3. Checksum

La tecnica del *Checksum* viene utilizzata per verificare l'integrità dei dati trasmessi; in particolare, durante la trasmissione, il mittente genera un valore numerico che rappresenta il contenuto di un singolo messaggio e lo include insieme ai dati da trasferire; in questo modo, il destinatario può ricalcolarlo e confrontarlo per rilevare eventuali errori o alterazioni avvenute durante il trasferimento.

A differenza del *Checksum* tradizionale usato in UDP, che calcola la somma dei blocchi a 16 bit applicandone il complemento a 1, nel nostro codice abbiamo utilizzato l'algoritmo CRC32 – *Cyclic Redundancy Check*; tale algoritmo elabora i dati di ingresso facendoli scorrere ciclicamente in una rete logica e li tratta come una sequenza binaria → divide i dati per un polinomio fisso e ottiene, come resto della divisione, un valore a 32 bit, più complesso e dettagliato rispetto a quello prodotto dal *Checksum* tradizionale; essendo a 32 bit, tale valore offre una maggiore capacità di rilevazione delle anomalie nella trasmissione e garantisce un controllo estremamente robusto dell'integrità dei dati.

```
[CHECKSUM]: Calculated checksum: 3539793434
[ACK MANAGER]: Sent packet: 31
[ACK MANAGER]: Packet 28 not retransmitted
[ACK MANAGER]: Received ACK: 28 | Timeout: 5534
[SLIDING WINDOW]: Sliding window moved: old send_base = 28, new send_base = 29
[ACK MANAGER]: SampleRTT:1212 | EstimatedRTT:2014 | DevRTT:880 | TIMEOUT:5534

[CHECKSUM]: Calculated checksum: 1057842832
[ACK MANAGER]: Sent packet: 32
[ACK MANAGER]: Packet 29 not retransmitted
[ACK MANAGER]: Received ACK: 29 | Timeout: 5212
[SLIDING WINDOW]: Sliding window moved: old send_base = 29, new send_base = 30
[ACK MANAGER]: SampleRTT:1267 | EstimatedRTT:1920 | DevRTT:823 | TIMEOUT:5212

[CHECKSUM]: Calculated checksum: 3956762076
[ACK MANAGER]: Sent packet: 33
[ACK MANAGER]: Packet 30 not retransmitted
[ACK MANAGER]: Received ACK: 30 | Timeout: 4881
[SLIDING WINDOW]: Sliding window moved: old send_base = 30, new send_base = 31
[ACK MANAGER]: SampleRTT:1259 | EstimatedRTT:1837 | DevRTT:761 | TIMEOUT:4881
```

Figura 9a: Checksum (Lato **Server**)

```
[ACK MANAGER]: Sliding window: base=30, last_pos=80
[ACK MANAGER]: Sent ACK for packet 30
[ACK MANAGER]: Written packet 30
[CHECKSUM]: Calculated checksum: 3539793434
[ACK MANAGER]: Received packet 31
[ACK MANAGER]: SampleRTT:515 | EstimatedRTT:662 | DevRTT:417 | TIMEOUT:2330

[ACK MANAGER]: Sliding window: base=31, last_pos=81
[ACK MANAGER]: Sent ACK for packet 31
[ACK MANAGER]: Written packet 31
[CHECKSUM]: Calculated checksum: 1057842832
[ACK MANAGER]: Received packet 32
[ACK MANAGER]: SampleRTT:567 | EstimatedRTT:650 | DevRTT:333 | TIMEOUT:1982

[ACK MANAGER]: Sliding window: base=32, last_pos=82
[ACK MANAGER]: Sent ACK for packet 32
[ACK MANAGER]: Written packet 32
[CHECKSUM]: Calculated checksum: 3956762076
[ACK MANAGER]: Received packet 33
[ACK MANAGER]: SampleRTT:307 | EstimatedRTT:607 | DevRTT:324 | TIMEOUT:1903
```

Figura 9b: Checksum (Lato **Client**)

7.6.4. CodeBase64

La tecnica *CodeBase64* converte i dati binari in una rappresentazione testuale tramite un algoritmo di codifica Base64 personalizzato; in questo modo, i dati risultano più facili da gestire in ambienti che richiedono formati testuali, mantenendone la compattezza; inoltre, tale tecnica facilita la trasmissione sicura dei dati ed aiuta a prevenire problemi di compatibilità tra diversi sistemi.

Nella fase di codifica (**Encode**), il contenuto di un file viene letto a blocchi e trasformato in una stringa composta da caratteri appartenenti ad un alfabeto Base64; l'alfabeto in questione è stato "shiftato" rispetto a quello standard, introducendo un ulteriore livello di personalizzazione; in questo modo, l'insieme dei caratteri utilizzati per rappresentare i dati risulta modificato, rendendo più difficile per un eventuale interferente decifrarne il contenuto senza conoscere lo schema di shift applicato.

Nella fase di decodifica (**Decode**), il processo si inverte → il file codificato viene letto e la sua rappresentazione testuale viene convertita nel formato binario originale; il file viene, quindi, ripristinato nella sua forma originaria, garantendone l'integrità e la fedeltà, pur essendo stato trasmesso in un formato più sicuro e standardizzato.

7.6.5. Cifrario Di Cesare

Il *Cifrario di Cesare* è una tecnica di cifratura a sostituzione molto semplice, in cui ogni lettera di un testo viene sostituita dalla lettera che si trova ad un numero fisso di posizioni più avanti nell'alfabeto → con uno shift di 3, "A" diventa "D", "B" diventa "E" e così via, con il riavvolgimento (*Wrap – Around*) alla fine dell'alfabeto; pur essendo facilmente decifrabile con metodi moderni, tale cifrario è storicamente rilevante ed ha fornito le basi per lo sviluppo della crittografia.
Nel modulo *CodeBase64*, questa tecnica viene applicata all'alfabeto Base64 standard, generandone una versione modificata tramite uno shift; in particolare, la funzione *generate_shifted_base64_alphabet* prende in input l'alfabeto originale e un valore di shift, normalizzandolo per rientrare nel range della lunghezza dell'alfabeto, ed applica uno shift ciclico → ad ogni indice *i* viene associato il carattere nella posizione *(i+shift) % length* (%

garantisce che lo spostamento sia circolare, riportando gli elementi all'inizio se superano la lunghezza massima); in questo modo si ottiene un nuovo alfabeto, che rende la codifica dei dati meno immediata nel riconoscimento, rendendo più difficile per un eventuale intercettatore decifrare i dati senza conoscere il parametro di shift applicato.

7.6.6. Timeout E RTT

Il meccanismo per il calcolo del *Timeout* si basa sulla misurazione dinamica del *Round – Trip – Time* (*RTT*) dei pacchetti, con l'obiettivo di adattare in tempo reale il tempo di attesa prima della ritrasmissione; il processo inizia con il calcolo del **Sample RTT** → tempo effettivo impiegato da un pacchetto per andare dal Mittente al Destinatario e tornare indietro, tramite *timestamp di arrivo (end) – timestamp di invio (start)*, convertendo i secondi in microsecondi e gestendo eventuali anomalie (ad esempio, quando i microsecondi dell'arrivo sono inferiori a quelli dell'invio); ottenuto il **Sample RTT**, viene aggiornato in maniera ricorsiva il valore dell'**Estimated RTT** tramite $Estimated\ RTT = (0.875 \times Estimated\ RTT) + (0.125 \times Sample\ RTT)$ → Media Pesata, che dà maggiore importanza alla stima precedente, rendendo il sistema meno sensibile a variazioni improvvise, ma comunque capace di adeguarsi gradualmente al tempo effettivo di andata e ritorno dei pacchetti; parallelamente, viene calcolata la **DevRTT** (Deviazione del RTT) tramite $DevRTT = (0.75 \times DevRTT) + (0.25 \times |Sample\ RTT - Estimated\ RTT|)$, che misura la dispersione dei tempi di risposta rispetto alla stima; infine, il **Timeout** viene calcolato tramite $Timeout = Estimated\ RTT + (4 \times DevRTT)$; questo approccio consente al sistema di adattarsi dinamicamente alle condizioni di rete → se i tempi di andata e ritorno dei pacchetti sono stabili, il timeout rimane contenuto, in presenza di variazioni o picchi di latenza, il timeout si adatta di conseguenza, riducendo al minimo sia le ritrasmissioni non necessarie sia i ritardi eccessivi prima di tentare un nuovo invio; in questo modo, si garantisce una gestione efficiente delle connessioni, anche in ambienti caratterizzati da elevata variabilità dei tempi di risposta.

7.7. Comando *quit*

Quando il Client decide di disconnettersi dalla comunicazione, invia il comando *quit* al Server, quest'ultimo riconosce il comando e avvia la procedura di disconnessione; l'operazione prevede → chiusura da parte del Server del canale di comunicazione (*Socket* associato al Client) → rilascio di tutte le risorse allocate per quel Client, il *thread* dedicato al Client in questione viene terminato, → aggiornamento della statistica degli utenti connessi rimuovendo il Client dall'elenco; una volta completate tutte queste operazioni, il Client riceve un messaggio di conferma della disconnessione e termina con successo.

8. Esempi Di Funzionamento

Qui di seguito alcuni esempi di comunicazione tra Client e Server, considerando una probabilità di perdita dei pacchetti del 20%; includeremo delle stampe esplicative per illustrare i tipi di messaggi scambiati.

8.1. Schermata Iniziale

All'avvio del sistema, nel lato Server, all'Utente viene presentata una schermata di Startup in cui è richiesto di inserire l'indirizzo IP (ad esempio, 127.0.0.1) e la porta di ascolto (ad esempio, 7777) del Server; successivamente, nel lato Client, viene visualizzata una schermata analoga, nella quale l'Utente deve inserire lo stesso indirizzo IP e la stessa porta per stabilire i parametri di connessione; instaurata la connessione, il sistema notifica l'avvenuto collegamento tra Client e Server e procede con la Multiplazione delle Porte → viene individuata la prima porta disponibile (ad esempio, 7778) a cui il Client viene reindirizzato.

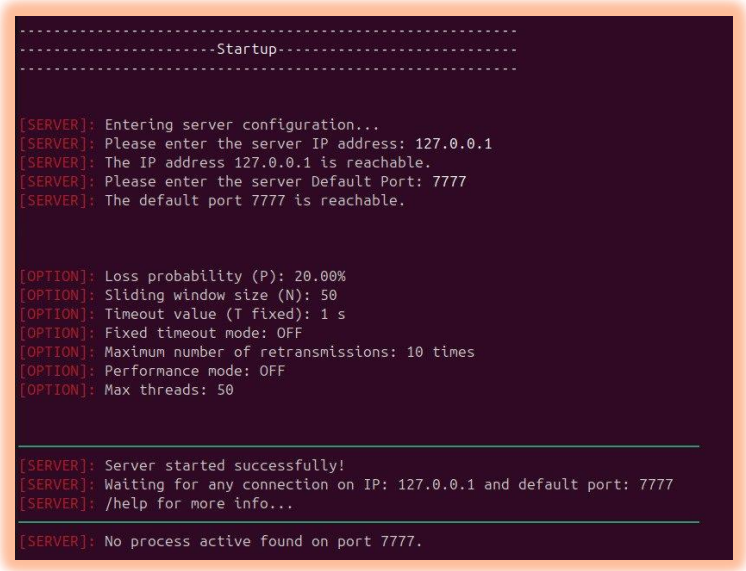


Figura 10a: Schermata di Startup (Lato **Server**)

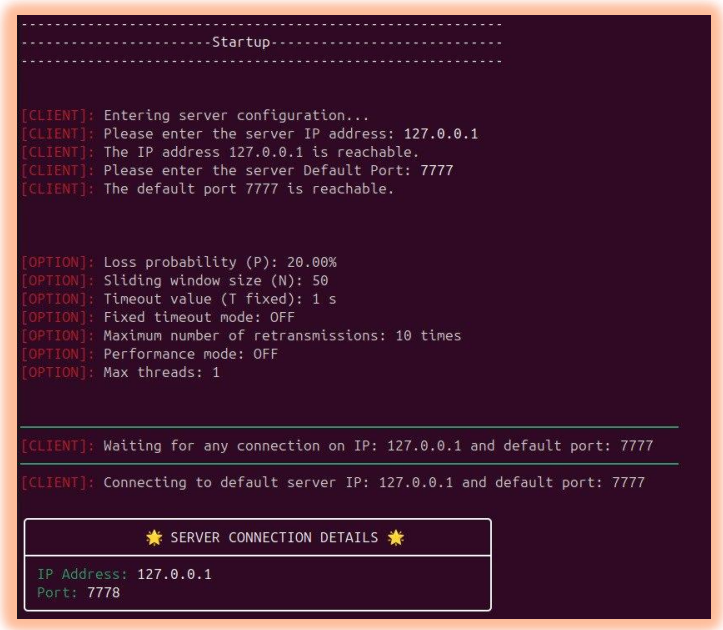


Figura 10b: Schermata di Startup (Lato **Client**)

```
[SERVER]: Multiplexing new connection on port: 7778
[MSGSYNC]: Succesfully SYNC
[CLIENT OBSERVER]: New Client-Server connection!
[CLIENT OBSERVER]: Client port: 7778, Socket: 4
[CLIENT OBSERVER]: Currently active clients: 1
[TIME LOGGER]: Connection time: 0.308603 s | Server Thread Index: 1
```

Figura 10c: Notifica di Connessione (Lato **Server**)

Una volta stabilita la connessione, nel lato Client, l'Utente visualizza un menu con tutti i comandi disponibili (*Figura 11b*); nel lato Server, digitando il comando */help*, vengono visualizzati tutti i comandi a disposizione; di seguito, mostriamo un esempio della schermata che apparirà durante l'esecuzione del software.

```
/help

🌟 AVAILABLE COMMANDS 🌟

1) /shutdown
   ➤ Close the UDP socket of the server
2) /delete <FileName>
   ➤ Delete a file from folder server_files
3) /list
   ➤ List all files in the server_files directory
4) /connections
   ➤ List all active connections
5) /disconnect <port>
   ➤ Disconnect a client on the specified port
   Warning: Use this command only to free
   a port if there are issues with the client
```

Figura 11a: Schermata Iniziale (Lato **Server**)

```
🌟 AVAILABLE COMMANDS 🌟

1) list
   ➤ Displays the files available on the server
2) get <FileName>
   ➤ Downloads the file <FileName> from the server
3) put <FileName>
   ➤ Uploads the file <FileName> to the server
4) modify <FileName>
   ➤ Edits the file <FileName> locally using Vim
5) delete <FileName>
   ➤ Delete the file <FileName> from the server
6) compress <FileName>
   ➤ Compress the file <FileName> locally
7) decompress <FileName>
   ➤ Decompress the file <FileName> locally
8) quit
   ➤ Ends the UDP connection with the server

ENTER YOUR DESIRED COMMAND:
```

Figura 11b: Schermata Iniziale (Lato **Client**)

Attenzione: se l'Utente non seleziona un comando tra quelli disponibili oppure lo inserisce in modo errato, riceverà un messaggio di avviso → *WARNING*.

```
ENTER YOUR DESIRED COMMAND: listt
Warning: invalid command!
```

Figura 12: Comando Non Valido

8.2. Messaggi Di Comando

Lato Server:

- **Comando /*shutdown*:** quando il Server utilizza tale comando, viene avviata una procedura per la chiusura ordinata del *Socket* UDP, interrompendo in maniera controllata l'accettazione di nuove comunicazioni → in quel momento, tutte le attività di rete in corso vengono terminate, garantendo il corretto rilascio delle risorse di sistema e prevenendo eventuali perdite di dati.

```
/shutdown
[SERVER]: Server shutting down...
[SERVER]: Server shutdown completed!
```

Figura 13: Comando */shutdown*

- **Comando /*delete*:** quando il Server utilizza il comando */delete* <*filename*>, viene avviata una procedura che consente di eliminare in maniera definitiva un file presente all'interno della cartella *server_files*; l'operazione prevede tre fasi:
  1. acquisizione del file specificato;
  2. cancellazione del file dal filesystem;
  3. liberazione dello spazio precedentemente occupato.

```
/delete BancaItalia.txt
[SERVER]: File server_files/BancaItalia.txt successfully deleted.
```

Figura 14: Comando */delete*

- **Comando /*list*:** quando il Server utilizza tale comando, viene generata una visualizzazione completa dell'elenco dei file attualmente presenti all'interno della directory *server_files*; tale comando, quindi, consente all'amministratore di ottenere una panoramica immediata e dettagliata dei contenuti disponibili, facilitando la gestione e l'organizzazione dei file → fornisce un'interfaccia semplice e diretta per monitorare lo stato della repository dei file del Server.

```
📂 LIST OF FILES 📂

file-sample1.doc
file-sample2.pdf
BancaItalia.txt
20mb.zip
file-sample5.jpeg
file-sample4.mp4
file-sample3.mp3
```

Figura 15: Comando */list*

- **Comando /*connections*:** quando il Server utilizza tale comando, viene generato un elenco dettagliato di tutte le connessioni attive nel sistema → utile per monitorare lo stato della rete e tenere traccia delle sessioni attive in tempo reale; tale comando consente, quindi, di visualizzare informazioni essenziali per ciascuna connessione, tra cui:
  - **Numero** di **Client** connessi;
  - **Porta** utilizzata;
  - ***Socket*** impiegato;
  - **Data** della connessione;
  - **Ora** della connessione.

```
/connections

🌐 ACTIVE CONNECTIONS 🌐

Connection: Port 7778, Socket 4, 2025-04-01 14:45:08
Connection: Port 7779, Socket 6, 2025-04-01 14:55:51
```

Figura 16: Comando */connections*

- **Comando /*disconnect*:** quando il Server utilizza il comando */disconnect* <*port*>, viene avviata una procedura mirata alla disconnessione di un Client dalla porta ad esso associata; tale comando consente, quindi, di interrompere in maniera selettiva la comunicazione con un Client che, a causa di instabilità o malfunzionamenti, necessita di essere isolato dal sistema.

```
/disconnect 7778
[SERVER]: Disconnected port 7778 with socket 4
```

Figura 17: Comando */disconnect*

<u>Lato Client</u>:

- **Comando *list*:** quando il Client utilizza tale comando, invia una richiesta al Server per ottenere l'elenco dei file disponibili per il download → attende di ricevere il numero di file presenti nel file *files_list.txt* → visualizza a schermo i nomi dei file; per elaborare la richiesta, il Server esamina il file *files_list.txt* → conta il numero di file disponibili → ne estrae i nomi → invia queste informazioni in due fasi:
  1. <u>numero totale di file disponibili</u>;
  2. <u>elenco dei nomi dei file</u>;

```
[CLIENT OBSERVER]:  Client port: 7778, Socket: 4
[COMMAND OBSERVER]: Command received: list
[LOGGER]:           Number of files: 7
[TIME LOGGER]:      executed in 0.001795 s
```

Figura 18a: Comando *list* (Lato **Server**)

📂 AVAILABLE FILES 📂	
7 Files	File Name
1	file-sample1.doc
2	file-sample2.pdf
3	BancaItalia.txt
4	20mb.zip
5	file-sample5.jpeg
6	file-sample4.mp4
7	file-sample3.mp3

Figura 18b: Comando *list* (Lato **Client**)

- **Comando *get*:** quando il Client utilizza il comando *get* <*filename*>, richiede al Server il file specificato nel comando; ricevuta la richiesta, il Server verifica se il file è disponibile nella cartella *server_files* → se presente, avvia il trasferimento dei dati, altrimenti, invia un messaggio di errore; il file ricevuto dal Client viene salvato nella cartella *Download*; qui di seguito tre possibili scenari:
  1. download avvenuto con successo;

```
[ACK MANAGER]: Received all ACK
[CHECKSUM]: Calculated checksum: 0
[ACK MANAGER]: Sent packet: 56
[ACK MANAGER]: Received ACK: 56
[FILE HANDLER]: Sent file: BancaItalia.txt
[TIME LOGGER]: executed in 2.248223 s
```

Figura 19a: Download Avvenuto con Successo (Lato **Server**)

```
[CODEBASE64]: File Download/BancaItalia.txt.txt decodificato in Download/BancaItalia.txt
[SERVER]: File Download/BancaItalia.txt.txt successfully deleted.
[FILE HANDLER]: I have received the file BancaItalia.txt
```

Figura 19b: Download Avvenuto con Successo (Lato **Client**)

  2. tentato download di un file non presente in *server_files*;

```
ENTER YOUR DESIRED COMMAND: get esempio.txt
The requested file is not in the list of available files on the server or is empty.
```

Figura 20: File Non Presente in *server_files*

  3. tentata esecuzione del comando *get* senza specificare un file;

```
ENTER YOUR DESIRED COMMAND: get
WARNING: You must enter a file.
```

Figura 21: Comando *get* Senza Nessun File

- **Comando *put*:** quando il Client utilizza il comando *put* <*filename*>, invia un file al

Server; il file viene prelevato dalla cartella *Upload* del Client e trasferito nella cartella *server_files* del Server; al termine dell'operazione, il file *files_list.txt*, che contiene l'elenco dei file disponibili, viene aggiornato inserendo il nome del nuovo file ricevuto; qui di seguito tre possibili scenari:

1. upload completato con successo;

```
[ACK MANAGER]: Download completed!
[CODEBASE64]: File server_files/BancaItalia.txt.txt decodificato in server_files/BancaItalia.txt
[SERVER]: File server_files/BancaItalia.txt.txt successfully deleted.
[FILE HANDLER]: Received file: BancaItalia.txt
[TIME LOGGER]: executed in 0.225759 s
```

Figura 22a: Upload Completato con Successo (Lato **Server**)

```
[ACK MANAGER]: All ACK received!
[CHECKSUM]: Calculated checksum: 0
[ACK MANAGER]: Sent packet: 56
[ACK MANAGER]: Received ACK: 56
[CLIENT]: File Upload/BancaItalia.txt.txt deleted successfully.
[FILE HANDLER]: File sent: BancaItalia.txt
```

Figura 22b: Upload Completato con Successo (Lato **Client**)

2. tentato upload di un File Non Presente in *Upload*;

```
ENTER YOUR DESIRED COMMAND: put esempio.txt
The file is not in the Upload folder or is empty.
```

Figura 23: File Non Presente in *Upload*

3. tentata esecuzione del comando *put* senza specificare un file;

```
ENTER YOUR DESIRED COMMAND: put
WARNING: You must enter a file.
```

Figura 24: Comando *put* Senza Nessun File

- **Comando *modify*:** quando il Client utilizza il comando *modify* <*filename*>, ha la possibilità di selezionare, tramite un menù interattivo, i tasti 1 o 2, che corrispondono alle cartelle *Download* e *Upload*; il Client, quindi, visualizzati i file presenti all'interno della directory selezionata, ne seleziona uno e lo apre per la modifica; il file viene aperto in un editor testuale, configurato in modalità sicura, permettendone una modifica protetta e controllata; qualora il file non dovesse esistere, l'Utente deve inserire il nome del file, che verrà creato automaticamente nella stessa directory.

```
[CLIENT OBSERVER]:  Client port: 7778, Socket: 4
[COMMAND OBSERVER]: Command received: modify
[TIME LOGGER]:      executed in 0.000081 s
```

Figura 25a: Comando *modify* (Lato **Server**)

```
ENTER YOUR DESIRED COMMAND: modify
[CLIENT OBSERVER]: Select a directory by digiting a number: [1.Download] [2.Upload]
[CLIENT OBSERVER]: Number Selected: 1
[CLIENT OBSERVER]: Directory selected: /home/DeS/Documents/Progetto_IIW/Client/Download
[CLIENT OBSERVER]: file-sample1.doc
[CLIENT OBSERVER]: file-sample2.pdf
[CLIENT OBSERVER]: BancaItalia.txt
[CLIENT OBSERVER]: 20mb.zip
[CLIENT OBSERVER]: file-sample5.jpeg
[CLIENT OBSERVER]: file-sample4.mp4
[CLIENT OBSERVER]: file-sample3.mp3
[CLIENT OBSERVER]: Insert the file name to modify or to be created if it does not exist: BancaItalia.txt
[CLIENT OBSERVER]: File editing complete.
```

Figura 25b: Comando *modify* (Lato **Client**)

- **Comando *delete*:** quando il Client utilizza il comando *delete <filename>*, richiede al Server l'eliminazione del file specificato; in particolare, il Server verifica se il file è presente nella cartella *server_files*, se esiste, procede con la sua rimozione, altrimenti invia un messaggio di errore al Client; al termine dell'operazione, il file *files_list.txt*, che contiene l'elenco dei file disponibili, viene aggiornato rimuovendo il nome del file di cui si è richiesta l'eliminazione.

```
[COMMAND OBSERVER]: Command received: delete BancaItalia.txt
[SERVER]: File server_files/BancaItalia.txt successfully deleted.
[FILE HANDLER]: Deleted file: BancaItalia.txt
[TIME LOGGER]:      executed in 0.000160 s
```

Figura 26a: Comando *delete* (Lato **Server**)

```
ENTER YOUR DESIRED COMMAND: delete BancaItalia.txt
[CLIENT]: File BancaItalia.txt deleted successfully.
```

Figura 26b: Comando *delete* (Lato **Client**)

- **Comando *compress*:** quando il Client utilizza il comando *compress <filename>*, ha la possibilità di selezionare, tramite un menù interattivo, i tasti 1 o 2, che corrispondono alle cartelle *Download* e *Upload*; il Client, quindi, visualizza i file presenti all'interno della directory selezionata, ne seleziona uno e procede alla compressione in formato *gzip*, generando un nuovo file con estensione *.gz* nella stessa directory; se l'operazione ha successo, il file originale viene rimosso e viene creato il file compresso, in caso di errore, viene mostrato un messaggio di errore all'Utente e l'operazione viene interrotta.

```
[COMMAND OBSERVER]: Command received: compress
[TIME LOGGER]:      executed in 0.000059 s
```

Figura 27a: Comando *compress* (Lato **Server**)

```
ENTER YOUR DESIRED COMMAND: compress
[CLIENT OBSERVER]: Select a directory by typing a number: [1.Download] [2.Upload]
[CLIENT OBSERVER]: Number Selected: 1
[CLIENT OBSERVER]: Directory selected: /home/DeS/Documents/Progetto_IIW/Client/Download
[CLIENT OBSERVER]: file-sample1.doc
[CLIENT OBSERVER]: file-sample2.pdf
[CLIENT OBSERVER]: BancaItalia.txt
[CLIENT OBSERVER]: 20mb.zip
[CLIENT OBSERVER]: file-sample5.jpeg
[CLIENT OBSERVER]: file-sample4.mp4
[CLIENT OBSERVER]: file-sample3.mp3
[CLIENT OBSERVER]: Insert the file name to be compressed: BancaItalia.txt
[CLIENT OBSERVER]: File '/home/DeS/Documents/Progetto_IIW/Client/Download/BancaItalia.txt' compressed successfully.
[CLIENT OBSERVER]: Decompressed file '/home/DeS/Documents/Progetto_IIW/Client/Download/BancaItalia.txt' deleted successfully.
```

Figura 27b: Comando *compress* (Lato **Client**)

- **Comando *decompress*:** quando il Client utilizza il comando *decompress* <*filename.gz*>, ha la possibilità di selezionare, tramite un menù interattivo, i tasti 1 o 2, che corrispondono alle cartelle *Download* e *Upload*; il Client, quindi, visualizza i file presenti all'interno della directory selezionata, ne seleziona uno e procede alla decompressione; in particolare, il contenuto del file selezionato, compresso con estensione *.gz*, viene estratto e salvato in un nuovo file privo dell'estensione *.gz* nella stessa directory; se l'operazione ha successo, il file compresso originale viene rimosso, in caso di errore, viene mostrato un messaggio di errore all'Utente e l'operazione viene interrotta.

```
[COMMAND OBSERVER]: Command received: decompress
[TIME LOGGER]:      executed in 0.000084 s
```

Figura 28a: Comando *decompress* (Lato **Server**)

```
ENTER YOUR DESIRED COMMAND: decompress
[CLIENT OBSERVER]: Select a directory by typing a number: [1.Download] [2.Upload]
[CLIENT OBSERVER]: Number Selected: 1
[CLIENT OBSERVER]: Directory selected: /home/DeS/Documents/Progetto_IIW/Client/Download
[CLIENT OBSERVER]: file-sample1.doc
[CLIENT OBSERVER]: file-sample2.pdf
[CLIENT OBSERVER]: BancaItalia.txt.gz
[CLIENT OBSERVER]: 20mb.zip
[CLIENT OBSERVER]: file-sample5.jpeg
[CLIENT OBSERVER]: file-sample4.mp4
[CLIENT OBSERVER]: file-sample3.mp3
[CLIENT OBSERVER]: Insert the file name to be decompressed: BancaItalia.txt.gz
[CLIENT OBSERVER]: File '/home/DeS/Documents/Progetto_IIW/Client/Download/BancaItalia.txt.gz' decompressed successfully.
[CLIENT OBSERVER]: Compressed file '/home/DeS/Documents/Progetto_IIW/Client/Download/BancaItalia.txt.gz' deleted successfully.
```

Figura 28b: Comando *decompress* (Lato **Client**)

- **Comando *quit*:** quando il Client inserisce tale comando, si prepara a chiudere la connessione; il Server, ricevuto il messaggio, disconnette il *thread* responsabile della gestione del Client e libera le risorse ad esso associate; il Client visualizza, poi, un messaggio di conferma della disconnessione da parte del Server.

```
[CLIENT OBSERVER]:  Client port: 7778, Socket: 4
[COMMAND OBSERVER]: Command received: quit
[CLIENT OBSERVER]:  Terminating session on port 7778 with socket 4
[CLIENT OBSERVER]:  Response received successfully. FIN
[CLIENT OBSERVER]:  Response received successfully. ACK
[CLIENT OBSERVER]:  Currently active clients: 0
[TIME LOGGER]:      executed in 1.031818 s
```

Figura 29a: Comando *quit* (Lato **Server**)

```
ENTER YOUR DESIRED COMMAND: quit
[CLIENT]: Notifying the server of system exit
[CLIENT]: Received successfully FIN+ACK
[CLIENT]: Closing session of this socket. Goodbye!
```

Figura 29b: Comando *quit* (Lato **Client**)

9. Limitazioni E Prospettive Di Miglioramento

L'applicazione Client – Server, pur fornendo una soluzione funzionale per la trasmissione dei file e la gestione delle connessioni, presenta alcune limitazioni intrinseche dovute alle scelte progettuali e alle tecnologie utilizzate.

9.1. Limitazioni Dovute all'Uso Di UDP

L'utilizzo del protocollo UDP, sebbene garantisca una bassa latenza ed un ridotto overhead, comporta l'assenza di meccanismi nativi per la garanzia di consegna, l'ordinamento dei pacchetti e la gestione degli errori a livello di trasporto → in caso di perdita di pacchetti o ritardi imprevedibili, il sistema deve necessariamente fare affidamento su meccanismi implementati a livello applicativo, come il protocollo Selective Repeat, per gestire le ritrasmissioni e garantire l'integrità dei dati trasmessi.

9.2. Restrizioni Sulla Dimensione Dei File E Sul Numero Massimo Di *Thread*

L'implementazione attuale potrebbe limitare la dimensione massima dei file trasferibili → file di grandi dimensioni richiedono una gestione più sofisticata della memoria e dei buffer; inoltre, il numero massimo di *thread* gestibili simultaneamente dipende dalle risorse hardware disponibili sul server; in condizioni di carico elevato, un numero elevato di connessioni concorrenti può causare un degrado delle prestazioni o portare all'esaurimento delle risorse di sistema, influendo negativamente sulla stabilità complessiva dell'applicazione.

9.3. Criticità, Bugs E Futuri Miglioramenti

Durante lo sviluppo ed i test del sistema sono emerse diverse criticità, che suggeriscono possibili miglioramenti futuri sia a livello funzionale che prestazionale, come:

- **Memory Leak** → si manifesta in condizioni di uso prolungato o durante trasferimenti multipli di file; se non risolto, può causare un degrado progressivo delle performance o, nel peggiore dei casi, il crash del programma;
- **Incoerenza Dei Comportamenti In Fase Di Compilazione Su Macchine Con Versioni Differenti Di Compilatori O Librerie** → ciò evidenzia la necessità di standardizzare rigorosamente l'ambiente di sviluppo tramite un sistema di build automatizzato o contenitori preconfigurati;
- **Utilizzo Della Funzione *printf*** → dal punto di vista prestazionale, su standard output, tale utilizzo introduce ritardi dovuti alla natura sincrona della funzione; si propone quindi l'adozione di una pipe ad essa dedicata per una gestione asincrona dell'output, migliorando la reattività del sistema e riducendo colli di bottiglia nella diagnostica;
- **Meccanismo Di Ripristino Della Connessione** → dal punto di vista della robustezza, in caso di chiusura improvvisa del programma, tale mancanza limiterebbe l'affidabilità del sistema; l'introduzione di una logica di riconnessione automatica e di ripresa del trasferimento a partire dall'ultimo pacchetto confermato ne riaumenterebbe l'affidabilità;
- **Meccanismo Di Autenticazione Tra Mittente E Destinatario** → dal punto di vista della sicurezza, tale mancanza lascia il sistema potenzialmente vulnerabile;

l'introduzione di un protocollo di autenticazione basato su token e l'adozione di una crittografia più efficiente per proteggere i dati scambiati ne ridurrebbe la vulnerabilità;

- **Disconnessioni Inattese** → legate a condizioni limite di rete o congestione momentanea; si propone quindi l'integrazione di un sistema di rilevamento e di gestione automatica di queste disconnessioni, con notifiche e tentativi di riconnessione.

Per le versioni future, inoltre, è auspicabile ottimizzare il protocollo di trasferimento per file di grandi dimensioni (nell'ordine dei gigabyte), tramite la parallelizzazione dei flussi o la compressione intelligente dei pacchetti; è necessario anche un processo di ottimizzazione generale del codice, volto a migliorarne la leggibilità, la modularità, la manutenibilità e la gestione delle risorse, accompagnato da una documentazione più completa.

10. Manuale Di Installazione, Configurazione Ed Esecuzione

Prerequisiti:

- **Sistema Operativo Linux** → Debian/Ubuntu, Arch, ecc…;
- **Connessione** a **Internet Attiva**;
- **Terminale Abilitato** → bash/zsh;

10.1. Esecuzione/Disinstallazione Del Codice Sul/Dal Proprio Dispositivo

Per effettuare l'installazione corretta del programma si prega di seguire questi semplici passaggi:

1. scaricare i file del progetto → scaricare tutti i file del progetto in una sola cartella (ad esempio, la cartella Download del proprio sistema → */home/nomeutente/Download/...*); i file richiesti sono:

- ***installation.sh***;
- ***tools_install.sh***;
- ***program_install.sh***;
- ***uninstall.sh***;
- ***Progetto_IIW.zip*** → contiene i file sorgente;

2. aprire il terminale → aprire il terminale ed entrare nella cartella dove si trovano i file del progetto scaricati precedentemente → *cd ~/Download*;
3. rendere eseguibili gli script → dare i permessi di esecuzione a tutti gli script *.sh* → *chmod +x*.sh*;
4. avviare l'installazione → lanciare lo script principale per installare il progetto → *./installation.sh*; lo script:

- ti chiederà se vuoi disinstallare una versione precedente del programma;
- ti chiederà se vuoi fare un'installazione standard o completa (con l'installazione degli strumenti necessari);

avviare la demo ed attendere l'apertura di due schermate bash per Client e Server:

- inserire IP e porta (default → **IP:** *127.0.0.1* e *7000* < **PORT** < *65535*) nel lato Server ed attendere il corretto avviamento;
- inserire stesso IP e porta del Server anche nel lato Client ed attendere il collegamento → comparsa della schermata iniziale;

qualora venissero riscontrati dei problemi → *./run_client.sh* e *./run_server.sh*.

Per effettuare la disinstallazione completa del programma si prega di eseguire questo semplice comando → *./uninstall.sh* oppure *./installation.sh*

10.2. Configurazione Dei Parametri (opzionale)

Si possono personalizzare alcuni parametri interni del programma, come:

- **N:** Dimensione della Sliding Window;
- **P:** Probabilità di Perdita;
- **T:** Timeout;
- **FIXEDTIMEOUT:** Flag di Abilitazione del Timeout Fisso;
- **MAX_*THREADS*:** Numero Massimo di Connessioni al Server;
- **PERFORMANCE:** Flag → ottimizza le prestazioni del sistema eliminando le notifiche

dei messaggi (ACK, Pacchetti, Timeout, ecc…);

- **MAXIMUM_ATTEMPT:** Numero Massimo di Tentativi per la Connessione del Client al Server;

tramite lo script *./config_macros.sh*; per lasciare invariato/inalterato un parametro → premere Invio; i file aggiornati sono:

- ***server/include/macros.h***;
- ***Client/include/macros.h***;
- **Finestra di Spedizione** → modificare la variabile W_Sender all'interno del file *Server.c*;
- **Finestra di Ricezione** → modificare la variabile W_Receiver all'interno del file *Client.c*;

10.3. Support

In caso di problemi, controllare i messaggi nel terminale; inoltre, se qualcosa non funziona, assicurarsi di:

- avere una connessione Internet attiva;
- trovarsi nella directory corretta;
- disporre dei permessi di esecuzione sugli script;
- avere installato le librerie necessarie per lo sviluppo in C;
- raggiungere nuovamente il progetto nella directory di sistema *~/local_bin/Progetto_IIW* e, eventualmente, eseguire *./run_demo.sh*, oppure ripetere la procedura di installazione;
- terminare prima il Client e successivamente il Server, così da garantire una chiusura corretta del progetto.

11. Ambiente Di Sviluppo E Librerie

11.1. Piattaforma Di Sviluppo

- Terminale **LINUX**
- **BLOCCO NOTE**
- **VSCODE**
- **VALGRIND**
- **GDB**

11.2. Librerie Utilizzate

- **#include <stdio.h>**
- **#include <string.h>**
- **#include <stdlib.h>**
- **#include <arpa/inet.h>**
- **#include <sys/socket.h>**
- **#include <fcntl.h>**
- **#include <limits.h>**
- **#include <wait.h>**
- **#include <sys/time.h>**
- **#include <stdbool.h>**
- **#include <pthread.h>**
- **#include <unistd.h>**
- **#include <errno.h>**
- **#include <memory.h>**
- **#include <dirent.h>**
- **#include <sys/select.h>**
- **#include <sys/stat.h>**
- **#include <signal.h>**
- **#include <sys/mman.h>**
- **#include <fcntl.h>**
- **#include <netinet/in.h>**
- **#include <zlib.h>**

11.3. Specifiche Di Sistema E Sviluppo (Linux – Fedora)

11.3.1. Sistema Operativo E Kernel

Linux fedora.local 6.13.9 – 200.fc41.x86_64 #1 SMP PREEMPT_DYNAMIC Sat Mar 29 01:29:31 UTC 2025 x86_64 GNU/Linux
Fedora release 41 (Forty One)
NAME = "Fedora Linux"
VERSION = "41 (Workstation Edition)"
RELEASE_TYPE = stable
ID = fedora
VERSION_ID = 41
VERSION_CODENAME = ""

PLATFORM_ID = "platform: f41"
PRETTY_NAME = "Fedora Linux 41 (Workstation Edition)"
ANSI_COLOR = "0; 38; 2; 60; 110; 180"
LOGO = fedora – logo – icon
CPE_NAME = "cpe:/o:fedoraproject:fedora:41"
DEFAULT_HOSTNAME = "fedora"
HOME_URL = "https://fedoraproject.org/"
DOCUMENTATION_URL = "https://docs.fedoraproject.org/en-US/fedora/f41/system-administrators-guide/"
SUPPORT_URL = "https://ask.fedoraproject.org/"
BUG_REPORT_URL = "https://bugzilla.redhat.com/"
REDHAT_BUGZILLA_PRODUCT = "Fedora"
REDHAT_BUGZILLA_PRODUCT_VERSION = 41
REDHAT_SUPPORT_PRODUCT = "Fedora"
REDHAT_SUPPORT_PRODUCT_VERSION = 41
SUPPORT_END = 2025 – 12 – 15
VARIANT = "Workstation Edition"
VARIANT_ID = workstation
Fedora release 41 (Forty One)
Fedora release 41 (Forty One)

11.3.2. Architettura CPU E Memoria

Architecture: x86_64
CPU op – mode(s): 32 – bit, 64 – bit
Address sizes: 48 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 6
On – line CPU(s) list: 0 – 5
Vendor ID: AuthenticAMD
Model name: AMD Ryzen 5 5500U with Radeon Graphics
CPU family: 23
Model: 104
Thread(s) per core: 2
Core(s) per socket: 3
Socket(s): 1
Stepping: 1
BogoMIPS: 4191.98
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid extd_apicid aperfmperf tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cmp_legacy cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoext ssbd ibrs ibpb stibp vmmcall fsgsbase bmi1 avx2 smep bmi2 rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 clzero xsaveerptr rdpru arat umip rdpid
Hypervisor vendor: Microsoft
Virtualization type: full

L1d cache: 96 KiB (3 instances)
L1i cache: 96 KiB (3 instances)
L2 cache: 1.5 MiB (3 instances)
L3 cache: 4 MiB (1 instance)
NUMA node(s): 1
NUMA node0 CPU(s): 0 – 5
Vulnerability Gather data sampling: Not affected
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf: Not affected
Vulnerability Mds: Not affected
Vulnerability Meltdown: Not affected
Vulnerability Mmio stale data: Not affected
Vulnerability Reg file data sampling: Not affected
Vulnerability Retbleed: Mitigation; untrained return thunk; SMT enabled with STIBP protection
Vulnerability Spec rstack overflow: Vulnerable: Safe RET, no microcode
Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2: Mitigation; Retpolines; IBPB conditional; STIBP always – on; RSB filling; PBRSB – eIBRS Not affected; BHI Not affected
Vulnerability Srbds: Not affected
Vulnerability Tsx async abort: Not affected

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	2.5Gi	165Mi	107Mi	1.5Gi	1.3Gi
Swap:	3.8Gi	563Mi	3.3Gi			

11.3.3. Versione Di GCC

gcc (GCC) 14.2.1 20250110 (Red Hat 14.2.1 – 7)
Copyright (C) 2024 Free Software Foundation, Inc
This is free software; see the source for copying conditions
There is NO warranty, not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE

11.3.4. Versione Di LIBC (glibc)

ldd (GNU libc) 2.40
Copyright (C) 2024 Free Software Foundation, Inc
This is free software; see the source for copying conditions
There is NO warranty, not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE
Written by Roland McGrath and Ulrich Drepper

11.3.5. Versione Di MAKE

GNU Make 4.4.1
Built for x86_64 – redhat – linux – gnu
Copyright (C) 1988 – 2023 Free Software Foundation, Inc

License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it
There is NO WARRANTY, to the extent permitted by law

11.3.6. Versione Di CMAKE (se presente)

cmake non installato

11.3.7. Versione Di GDB (Debugger)

GNU gdb (Fedora Linux) 16.2 – 2.fc41
Copyright (C) 2024 Free Software Foundation, Inc
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it
There is NO WARRANTY, to the extent permitted by law

11.4. Specifiche Di Sistema E Sviluppo (Linux – Ubuntu)

11.4.1. Sistema Operativo E Kernel

Linux DeS 6.11.0 – 21 – generic #21~24.04.1 – Ubuntu SMP PREEMPT_DYNAMIC Mon Feb 24 16:52:15 UTC 2 x86_64 x86_64 x86_64 GNU/Linux

Distributor ID:	Ubuntu
Description:	Ubuntu 24.04.2 LTS
Release:	24.04
Codename:	noble

11.4.2. Architettura CPU E Memoria

Architecture:	x86_64
CPU op – mode(s):	32 – bit, 64 – bit
Address sizes:	39 bits physical, 48 bits virtual
Byte Order:	Little Endian
CPU(s):	5
On – line CPU(s) list:	0 – 4
Vendor ID:	GenuineIntel
Model name:	12th Gen Intel(R) Core(TM) i7 – 12650H
CPU family:	6
Model:	154
Thread(s) per core:	1
Core(s) per socket:	5
Socket(s):	1
Stepping:	3
BogoMIPS:	5375.72
Flags:	fpu vme de pse tsc msr pae mce cx8 apic sep

mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx rdtscp lm constant_tsc rep_good nopl xtopology nonstop_tsc cpuid tsc_known_freq pni pclmulqdq ssse3 cx16 sse4_1 sse4_2 movbe popcnt aes rdrand hypervisor lahf_lm abm 3dnowprefetch ibrs_enhanced fsgsbase bmi1 bmi2 invpcid rdseed adx clflushopt sha_ni arat md_clear

flush_l1d arch_capabilities
Hypervisor vendor: KVM
Virtualization type: full
L1d cache: 240 KiB (5 instances)
L1i cache: 160 KiB (5 instances)
L2 cache: 6.3 MiB (5 instances)
L3 cache: 120 MiB (5 instances)
NUMA node(s): 1
NUMA node0 CPU(s): 0 – 4
Vulnerability Gather data sampling: Not affected
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf: Not affected
Vulnerability Mds: Not affected
Vulnerability Meltdown: Not affected
Vulnerability Mmio stale data: Not affected
Vulnerability Reg file data sampling: Vulnerable: No microcode
Vulnerability Retbleed: Mitigation; Enhanced IBRS
Vulnerability Spec rstack overflow: Not affected
Vulnerability Spec store bypass: Vulnerable
Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2: Mitigation; Enhanced / Automatic IBRS; RSB filling; PBRSB – eIBRS SW sequence; BHI SW loop, KVM SW loop
Vulnerability Srbds: Not affected
Vulnerability Tsx async abort: Not affected

	total	used	free	shared	buff/cache	available
Mem:	5.6Gi	1.9Gi	2.4Gi	142Mi	1.8Gi	3.8Gi
Swap:	0B	0B	0B			

11.4.3. Versione Di GCC

gcc (GCC) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc
This is free software; see the source for copying conditions
There is NO warranty, not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE

11.4.4. Versione Di LIBC (glibc)

ldd (Ubuntu GLIBC 2.39 – 0ubuntu8.4) 2.39
Copyright (C) 2024 Free Software Foundation, Inc
This is free software; see the source for copying conditions
There is NO warranty, not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE
Written by Roland McGrath and Ulrich Drepper

11.4.5. Versione Di MAKE

GNU Make 4.3

Built for x86_64 – pc – linux – gnu
Copyright (C) 1988 – 2020 Free Software Foundation, Inc
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it
There is NO WARRANTY, to the extent permitted by law

11.4.6. Versione Di CMAKE (se presente)

cmake non installato

11.4.7. Versione Di GDB (Debugger)

GNU gdb (Ubuntu 15.0.50.20240403 – 0ubuntu1) 15.0.50.20240403 – git
Copyright (C) 2024 Free Software Foundation, Inc
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it
There is NO WARRANTY, to the extent permitted by law

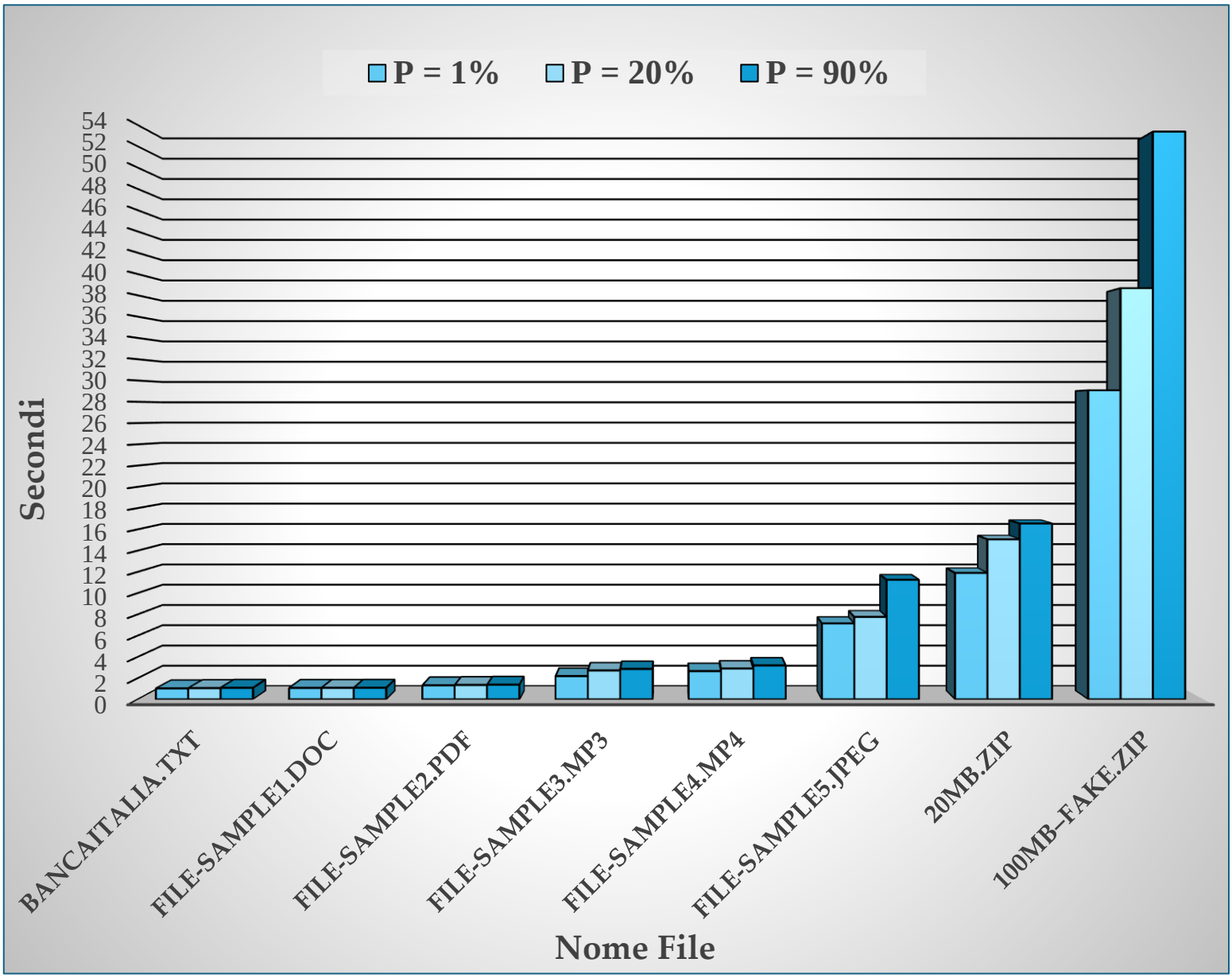
12. Valutazioni Prestazionali

Per effettuare le valutazioni prestazionali del progetto, è stata adottata una modalità denominata *Performance*, modificabile nel file *macros.h*, per la trasmissione dei pacchetti; una volta attivata → impostata su 1, il sistema disabilita la visualizzazione e l'invio dei messaggi diagnostici (ad esempio, *Sample RTT*, *Timeout*, *Checksum*, *SendBase*, ecc…), concentrando così le risorse esclusivamente sulla trasmissione dei dati; tale modalità, inoltre, consente di ridurre significativamente il sovraccarico computazionale e di I/O, migliorando il throughput e la latenza complessiva del sistema; in questo modo, le valutazioni prestazionali si focalizzano sulla qualità della trasmissione, garantendo risultati più affidabili e rappresentativi delle condizioni operative reali, soprattutto in contesti in cui le prestazioni sono un requisito fondamentale; per le valutazioni è stato impiegato anche un file di 100 MB → *100mb–fake.zip*, che, per motivi di dimensione e per limitazioni relative allo spazio occupato nella cartella dei file di esempio, non è stato incluso nel pacchetto *file–sample* inviato.

Nelle pagine a seguire sono riportate le tabelle ed i relativi grafici contenenti i parametri utilizzati per le valutazioni prestazionali.

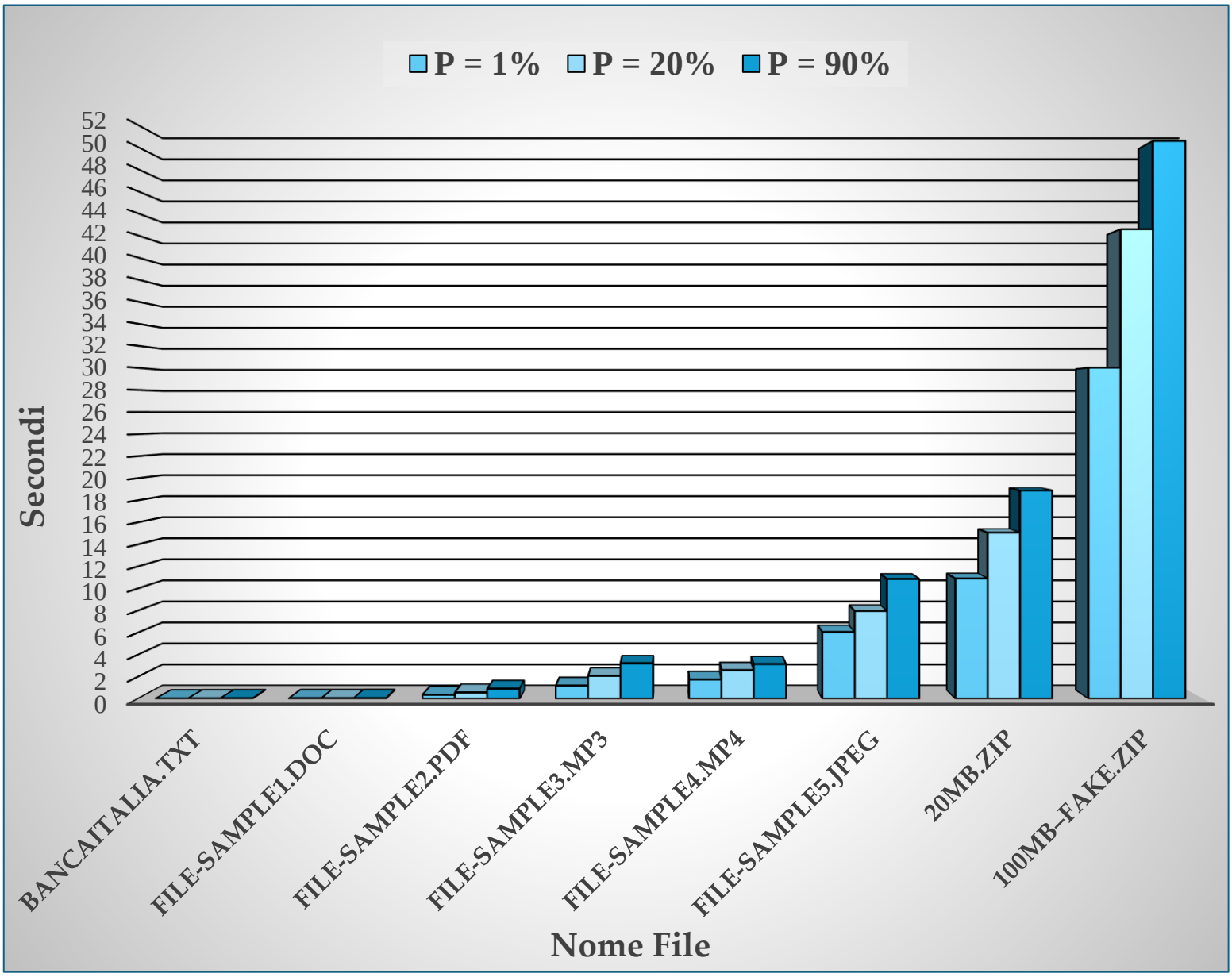
12.1. Tabella 1 – *get* File: **N**=100; **WR/WS**=50; **T** adattivo; **P** (Probabilità di Perdita)

	Peso File	P=1%	P=20%	P=90%
BancaItalia.txt	62.3 KB	1.026567 s	1.050620 s	1.080851 s
file–sample1.doc	100.4 KB	1.069771 s	1.073799 s	1.081200 s
file–sample2.pdf	1.3 MB	1.315222 s	1.344294 s	1.372438 s
file–sample3.mp3	3.9 MB	2.183988 s	2.712680 s	2.852461 s
file–sample5.jpeg	5.6 MB	2.648500 s	2.894096 s	3.187256 s
20mb.zip	20 MB	7.145876 s	7.754091 s	11.251302 s
file–sample4.mp4	38.2 MB	11.890698 s	15.056499 s	16.545158 s
100mb–fake.zip	104.7 MB	29.080291 s	38.678380 s	53.383763 s



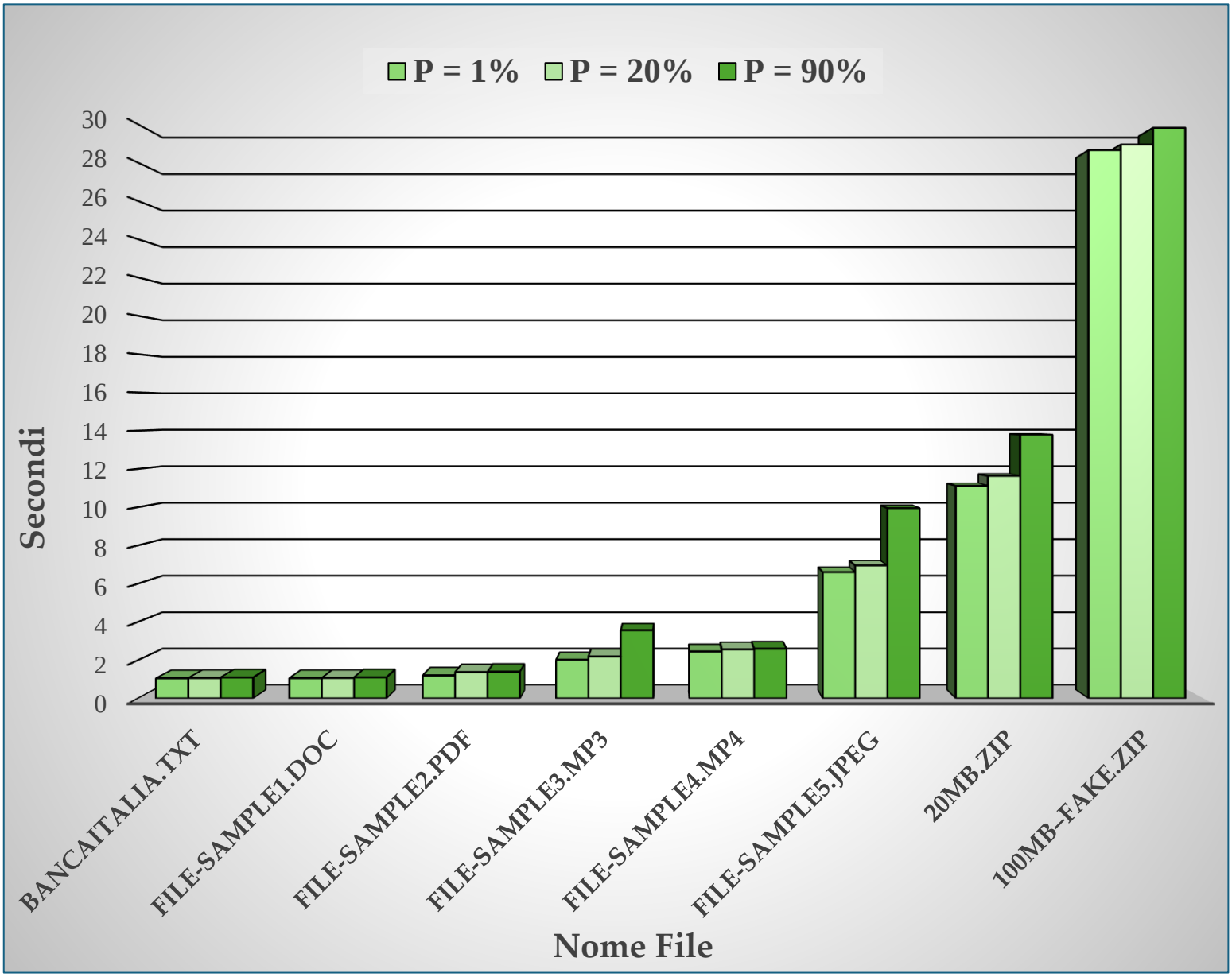
12.2. Tabella 2 – *put* File: **N**=100; **WR/WS**=50; **T** adattivo; **P** (Probabilità di Perdita)

	Peso File	P=1%	P=20%	P=90%
BancaItalia.txt	62.3 KB	0.022745 s	0.033222 s	0.043028 s
file–sample1.doc	100.4 KB	0.047088 s	0.051721 s	0.053963 s
file–sample2.pdf	1.3 MB	0.361238 s	0.569437 s	0.915341 s
file–sample3.mp3	3.9 MB	1.194094 s	2.079348 s	3.221847 s
file–sample5.jpeg	5.6 MB	1.750529 s	2.598399 s	3.151927 s
20mb.zip	20 MB	6.057677 s	7.959632 s	10.867920 s
file–sample4.mp4	38.2 MB	10.912477 s	15.065890 s	18.895345 s
100mb–fake.zip	104.7 MB	30.011202 s	42.553839 s	50.546644 s



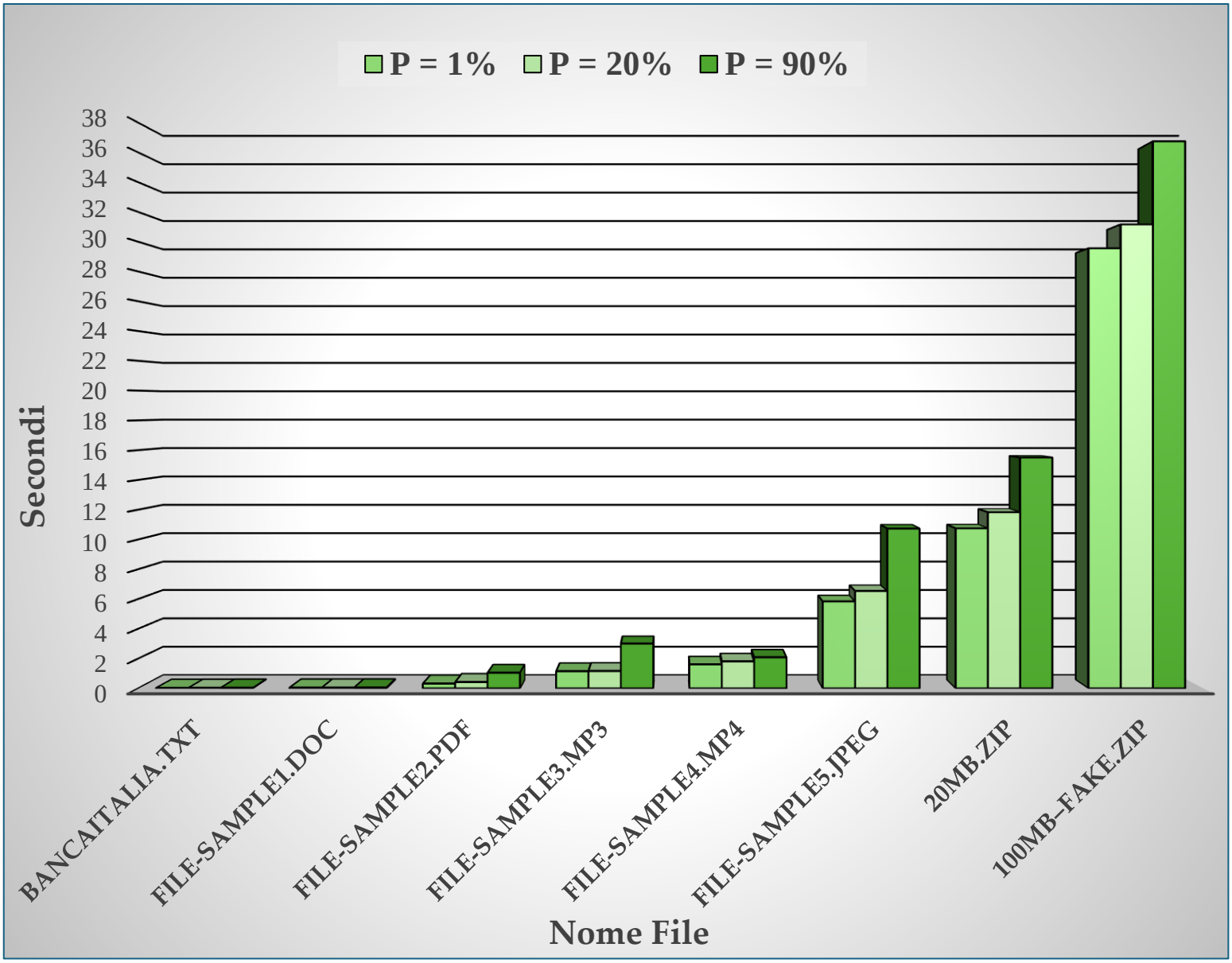
12.3. Tabella 3 – *get* File: **N**=50; **WR/WS**=25; **T** adattivo; **P** (Probabilità di Perdita)

	Peso File	P=1%	P=20%	P=90%
BancaItalia.txt	62.3 KB	1.052320 s	1.064275 s	1.094634 s
file–sample1.doc	100.4 KB	1.057905 s	1.065308 s	1.100494 s
file–sample2.pdf	1.3 MB	1.200390 s	1.372160 s	1.391611 s
file–sample3.mp3	3.9 MB	2.015798 s	2.187588 s	3.562113 s
file–sample5.jpeg	5.6 MB	2.449172 s	2.562757 s	2.593490 s
20mb.zip	20 MB	6.603858 s	6.951038 s	9.945449 s
file–sample4.mp4	38.2 MB	11.124418 s	11.630665 s	13.790192 s
100mb–fake.zip	104.7 MB	28.646804 s	28.938565 s	29.811613 s



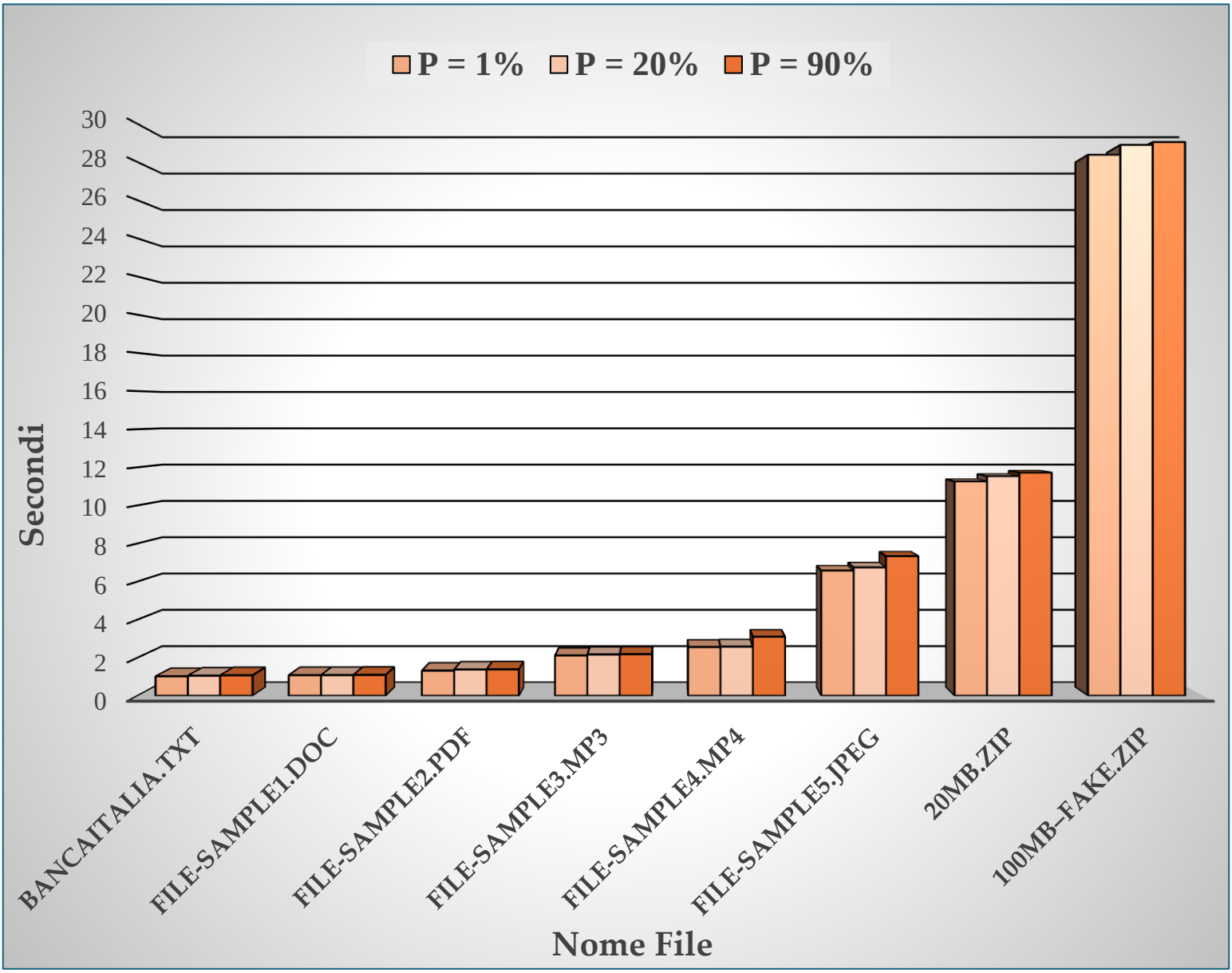
12.4. Tabella 4 – *put* File: **N**=50; **WR/WS**=25; **T** adattivo; **P** (Probabilità di Perdita)

	Peso File	P=1%	P=20%	P=90%
BancaItalia.txt	62.3 KB	0.027898 s	0.029690 s	0.035859 s
file–sample1.doc	100.4 KB	0.042320 s	0.046817 s	0.051925 s
file–sample2.pdf	1.3 MB	0.319185 s	0.419947 s	1.051532 s
file–sample3.mp3	3.9 MB	1.154620 s	1.166342 s	3.008051 s
file–sample5.jpeg	5.6 MB	1.611754 s	1.811747 s	2.086266 s
20mb.zip	20 MB	5.839972 s	6.541690 s	10.732280 s
file–sample4.mp4	38.2 MB	10.749228 s	11.831304 s	15.504793 s
100mb–fake.zip	104.7 MB	29.553917 s	31.158989 s	36.740815 s



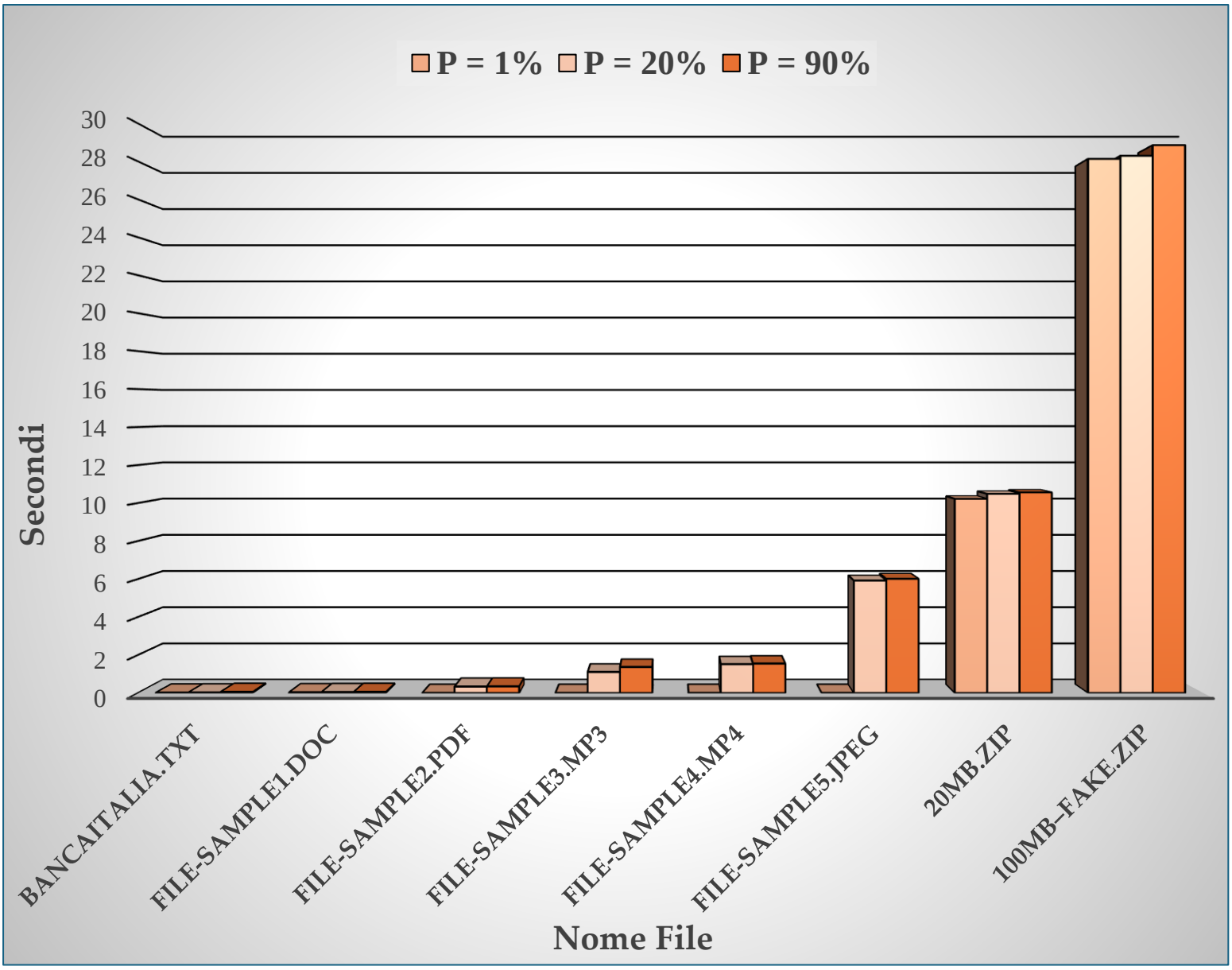
12.5. Tabella 5 – *get* File: **N**=20; **WR/WS**=10; **T** adattivo; **P** (Probabilità di Perdita)

	Peso File	P=1%	P=20%	P=90%
BancaItalia.txt	62.3 KB	1.020899 s	1.049266 s	1.072103 s
file–sample1.doc	100.4 KB	1.077570 s	1.080222 s	1.090835 s
file–sample2.pdf	1.3 MB	1.317143 s	1.375078 s	1.382542 s
file–sample3.mp3	3.9 MB	2.112676 s	2.166897 s	2.179163 s
file–sample5.jpeg	5.6 MB	2.557418 s	2.576820 s	3.098891 s
20mb.zip	20 MB	6.574038 s	6.743611 s	7.329410 s
file–sample4.mp4	38.2 MB	11.257360 s	11.537311 s	11.719104 s
100mb–fake.zip	104.7 MB	28.382096 s	28.898324 s	29.046313 s



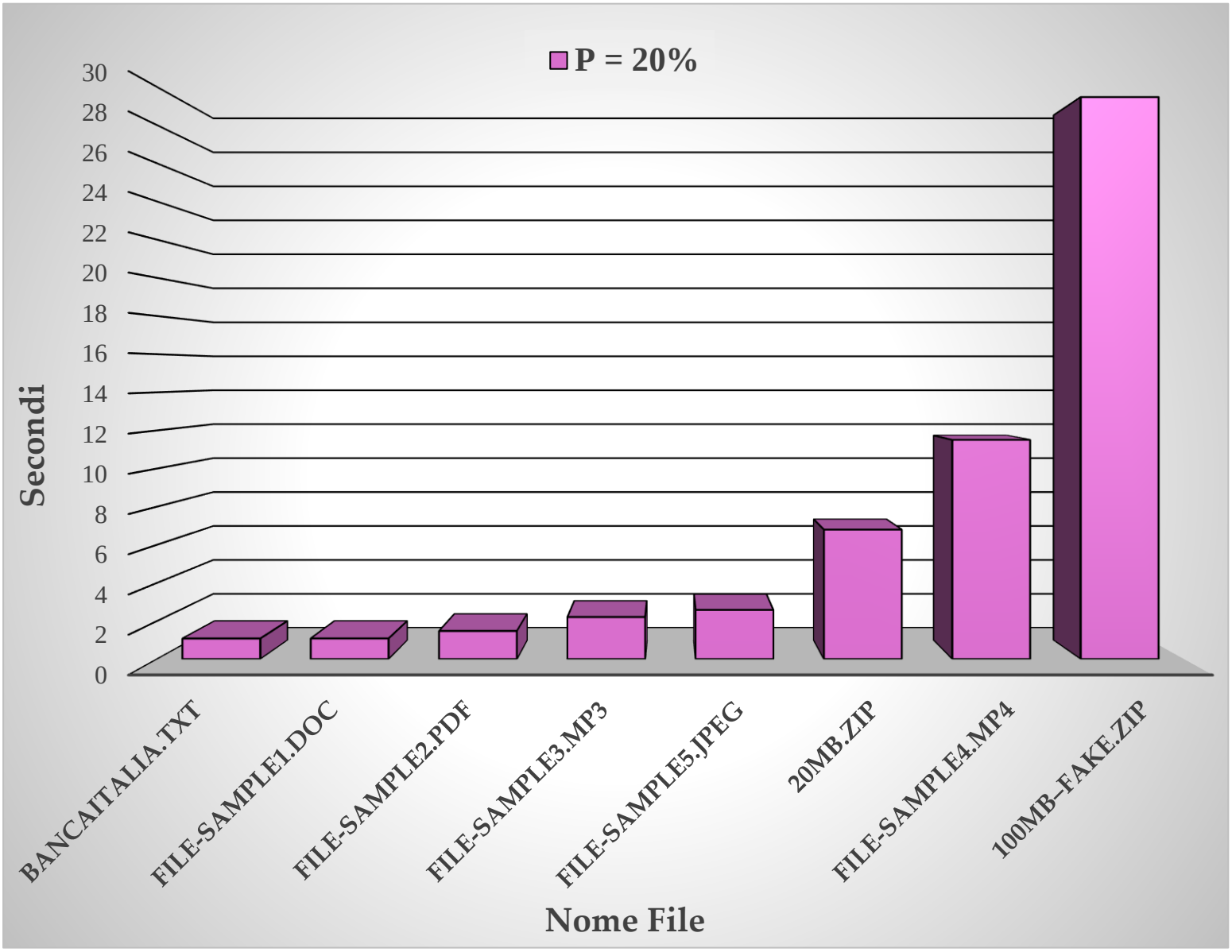
12.6. Tabella 6 – *put* File: **N**=20; **WR/WS**=10; **T** adattivo; **P** (Probabilità di Perdita)

	Peso File	P=1%	P=20%	P=90%
BancaItalia.txt	62.3 KB	0.018865 s	0.022964 s	0.052882 s
file–sample1.doc	100.4 KB	0.033589 s	0.040690 s	0.050678 s
file–sample2.pdf	1.3 MB	0.323528 s	0.336942 s	0.344361 s
file–sample3.mp3	3.9 MB	1.052315 s	1.103709 s	1.363591 s
file–sample5.jpeg	5.6 MB	1.401378 s	1.513852 s	1.544801 s
20mb.zip	20 MB	5.783741 s	5.916414 s	6.010931 s
file–sample4.mp4	38.2 MB	10.221226 s	10.483540 s	10.559499 s
100mb–fake.zip	104.7 MB	28.130492 s	28.293019 s	28.853656 s



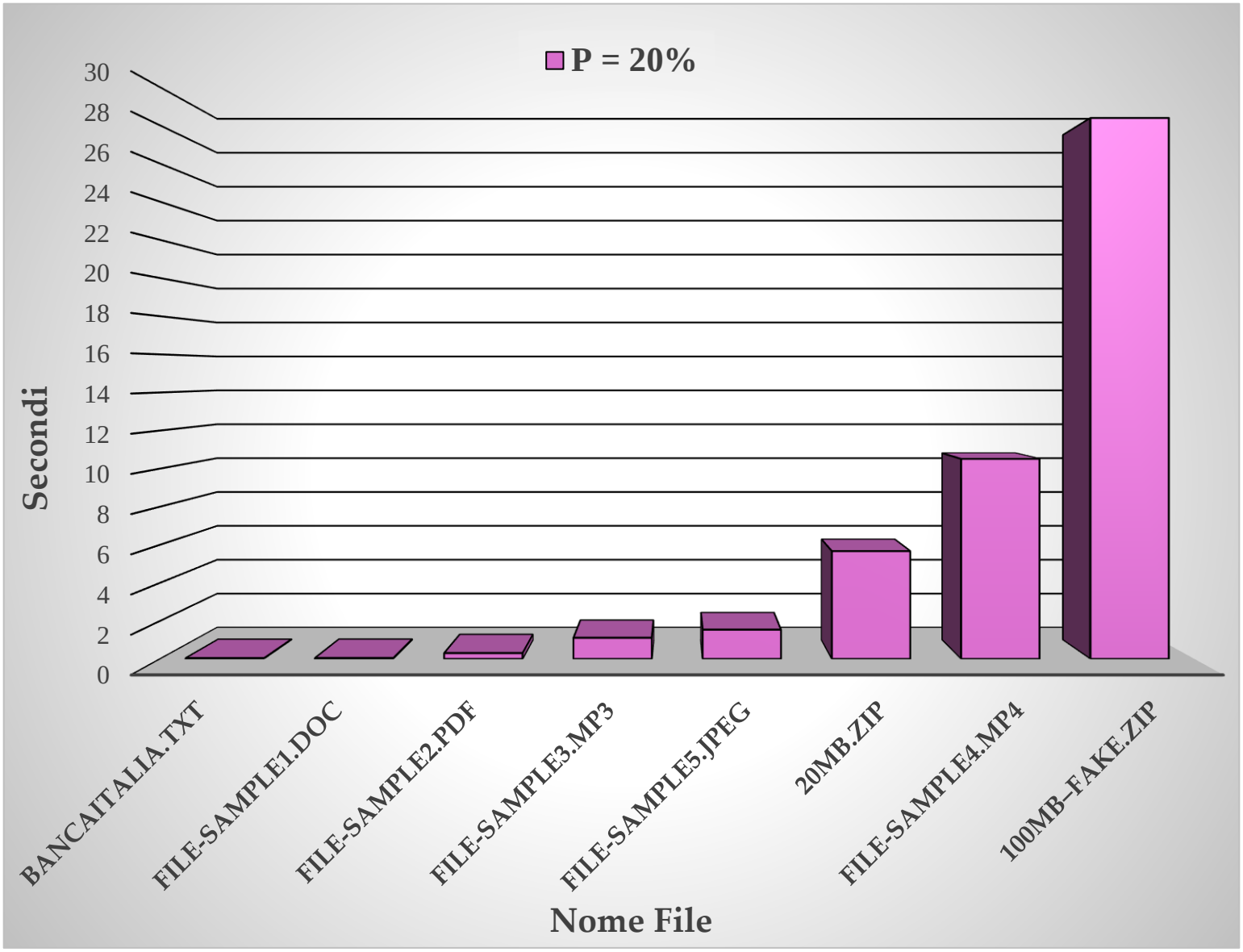
12.7. Tabella 7 – *get* File: **N**=50; **WR/WS**=25; **T**=1 s; **P** (Probabilità di Perdita)

	Peso File	P=20%
BancaItalia.txt	62.3 KB	1.075530 s
file–sample1.doc	100.4 KB	1.077963 s
file–sample2.pdf	1.3 MB	1.471838 s
file–sample3.mp3	3.9 MB	2.202055 s
file–sample5.jpeg	5.6 MB	2.576091 s
20mb.zip	20 MB	6.791989 s
file–sample4.mp4	38.2 MB	11.503100 s
100mb–fake.zip	104.7 MB	29.472883 s



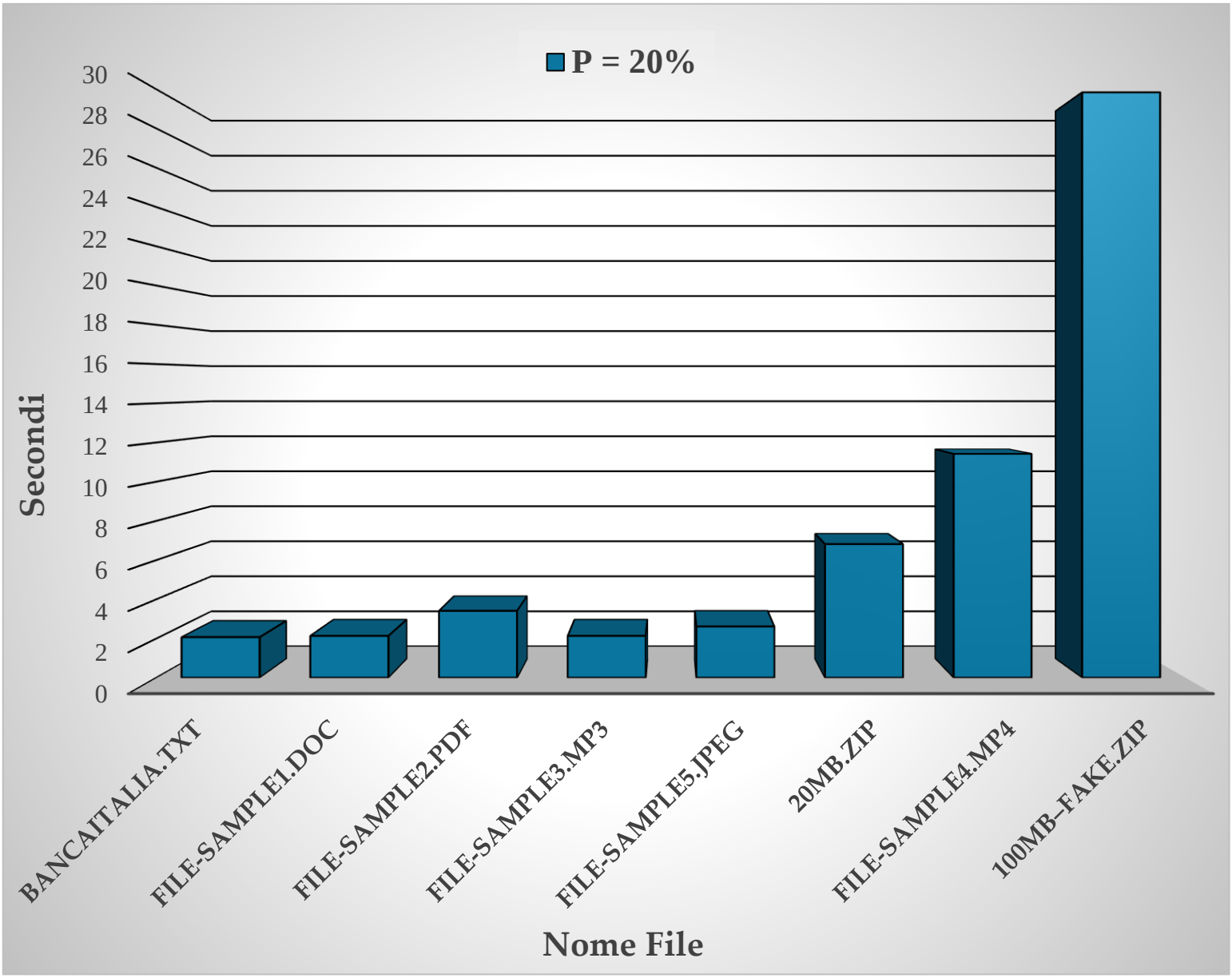
12.8. Tabella 8 – *put* File: **N**=50; **WR/WS**=25; **T**=1 s; **P** (Probabilità di Perdita)

	Peso File	P=20%
BancaItalia.txt	62.3 KB	0.035808 s
file–sample1.doc	100.4 KB	0.041098 s
file–sample2.pdf	1.3 MB	0.304460 s
file–sample3.mp3	3.9 MB	1.106999 s
file–sample5.jpeg	5.6 MB	1.533706 s
20mb.zip	20 MB	5.660746 s
file–sample4.mp4	38.2 MB	10.510305 s
100mb–fake.zip	104.7 MB	28.388513 s



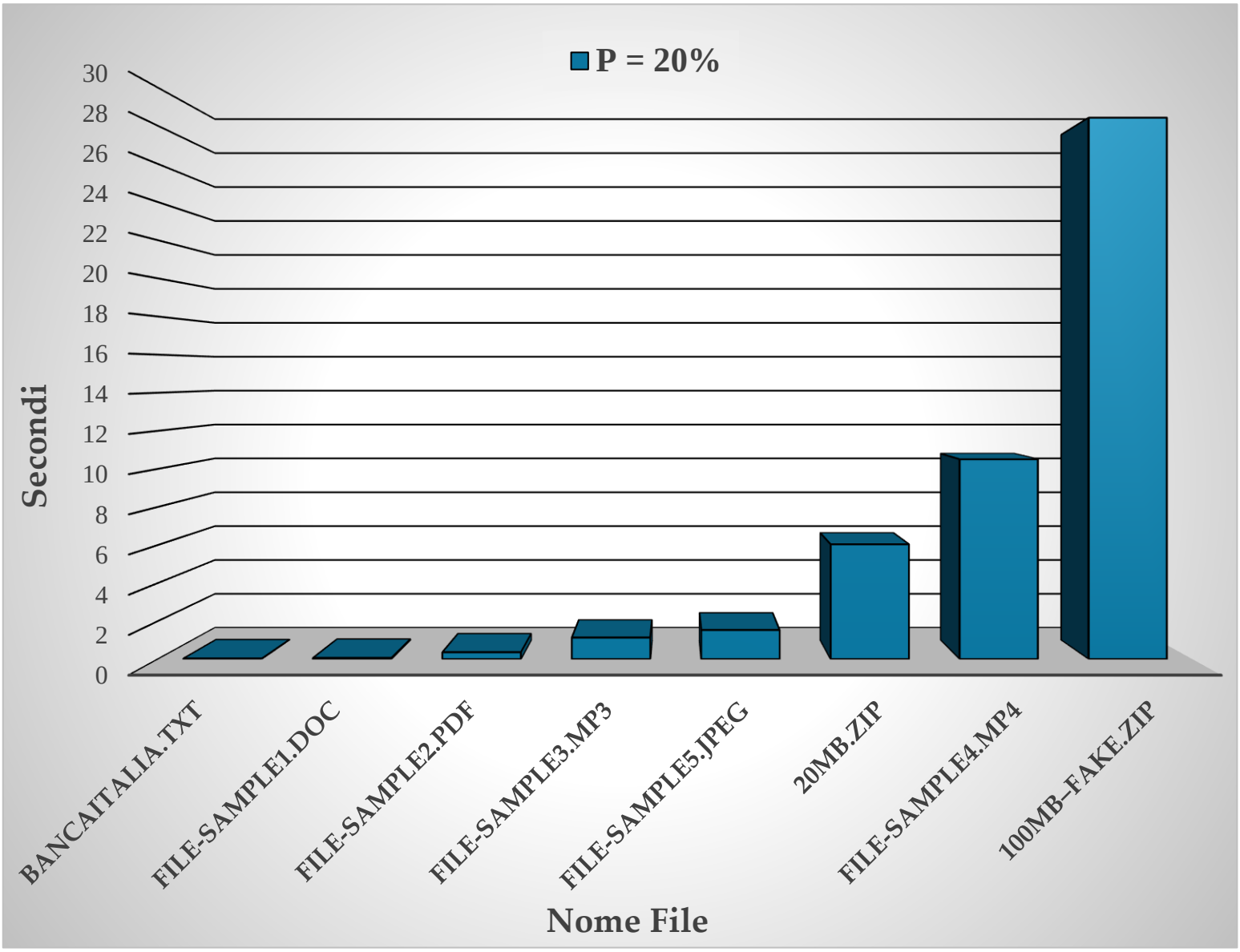
12.9. Tabella 9 – *get* File: **N**=50; **WR/WS**=25; **T**=2 s; **P** (Probabilità di Perdita)

	Peso File	P=20%
BancaItalia.txt	62.3 KB	2.066970 s
file–sample1.doc	100.4 KB	2.132804 s
file–sample2.pdf	1.3 MB	3.410928 s
file–sample3.mp3	3.9 MB	2.132962 s
file–sample5.jpeg	5.6 MB	2.611506 s
20mb.zip	20 MB	6.814151 s
file–sample4.mp4	38.2 MB	11.421628 s
100mb–fake.zip	104.7 MB	29.841551 s



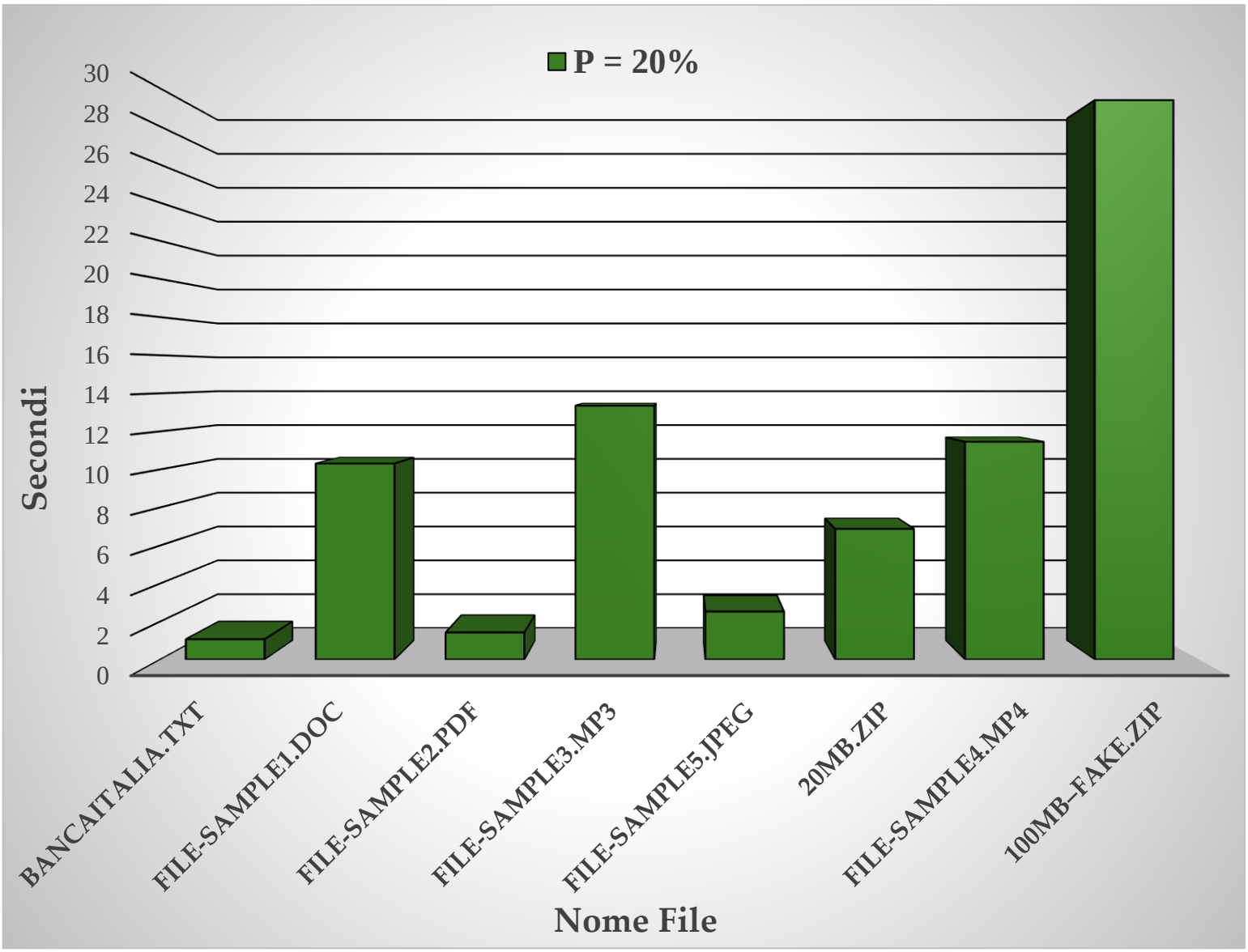
12.10. Tabella 10 – *put* File: **N**=50; **WR/WS**=25; **T**=2 s; **P** (Probabilità di Perdita)

	Peso File	P=20%
BancaItalia.txt	62.3 KB	0.026070 s
file–sample1.doc	100.4 KB	0.053675 s
file–sample2.pdf	1.3 MB	0.358935 s
file–sample3.mp3	3.9 MB	1.129996 s
file–sample5.jpeg	5.6 MB	1.526419 s
20mb.zip	20 MB	6.033002 s
file–sample4.mp4	38.2 MB	10.503240 s
100mb–fake.zip	104.7 MB	28.423003 s



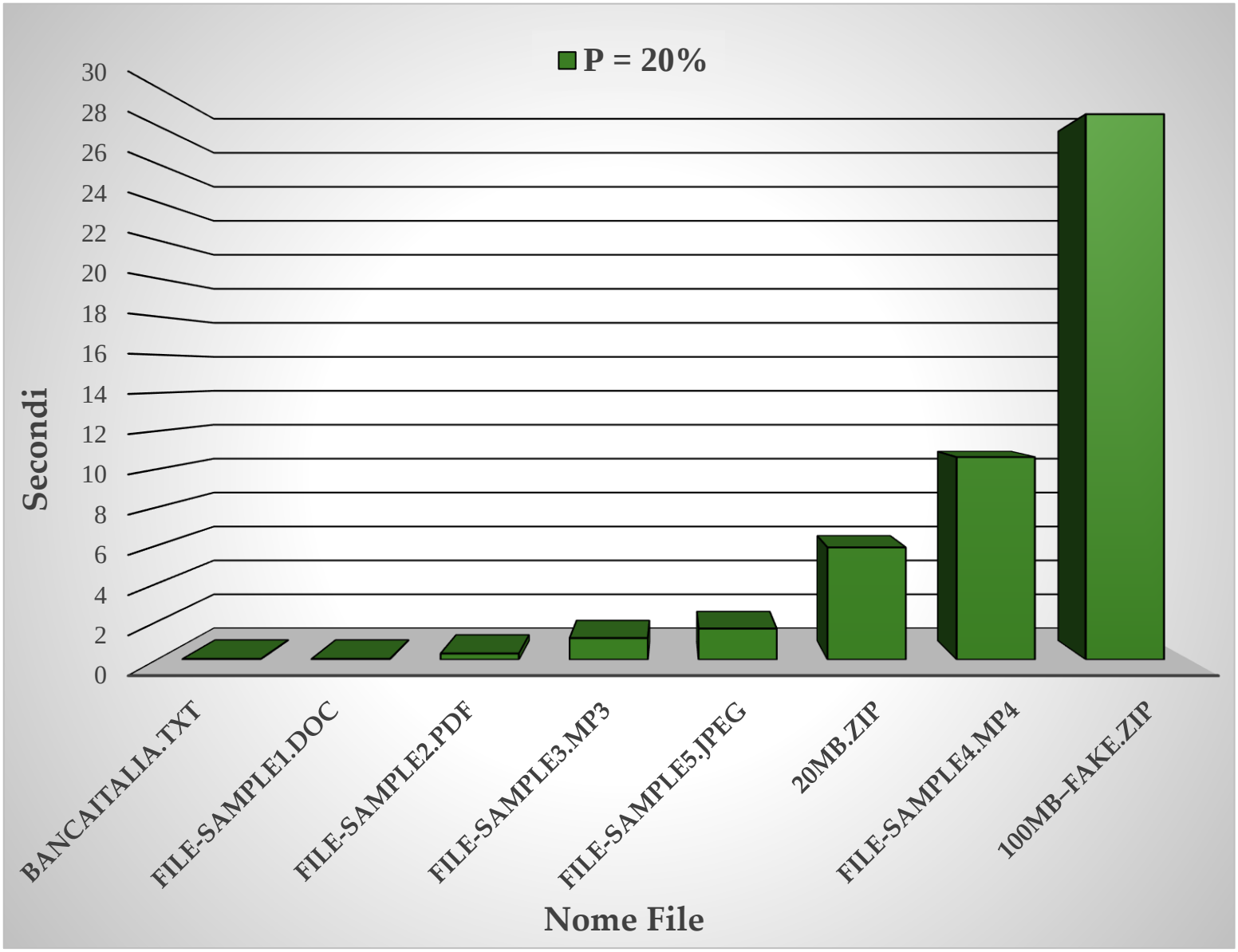
12.11. Tabella 11 – *get* File: N=50; WR/WS=25; T=10 s; P (Probabilità di Perdita)

	Peso File	P=20%
BancaItalia.txt	62.3 KB	1.066244 s
file–sample1.doc	100.4 KB	10.308272 s
file–sample2.pdf	1.3 MB	1.421335 s
file–sample3.mp3	3.9 MB	13.349668 s
file–sample5.jpeg	5.6 MB	2.521831 s
20mb.zip	20 MB	6.871468 s
file–sample4.mp4	38.2 MB	11.449796 s
100mb–fake.zip	104.7 MB	29.381306 s



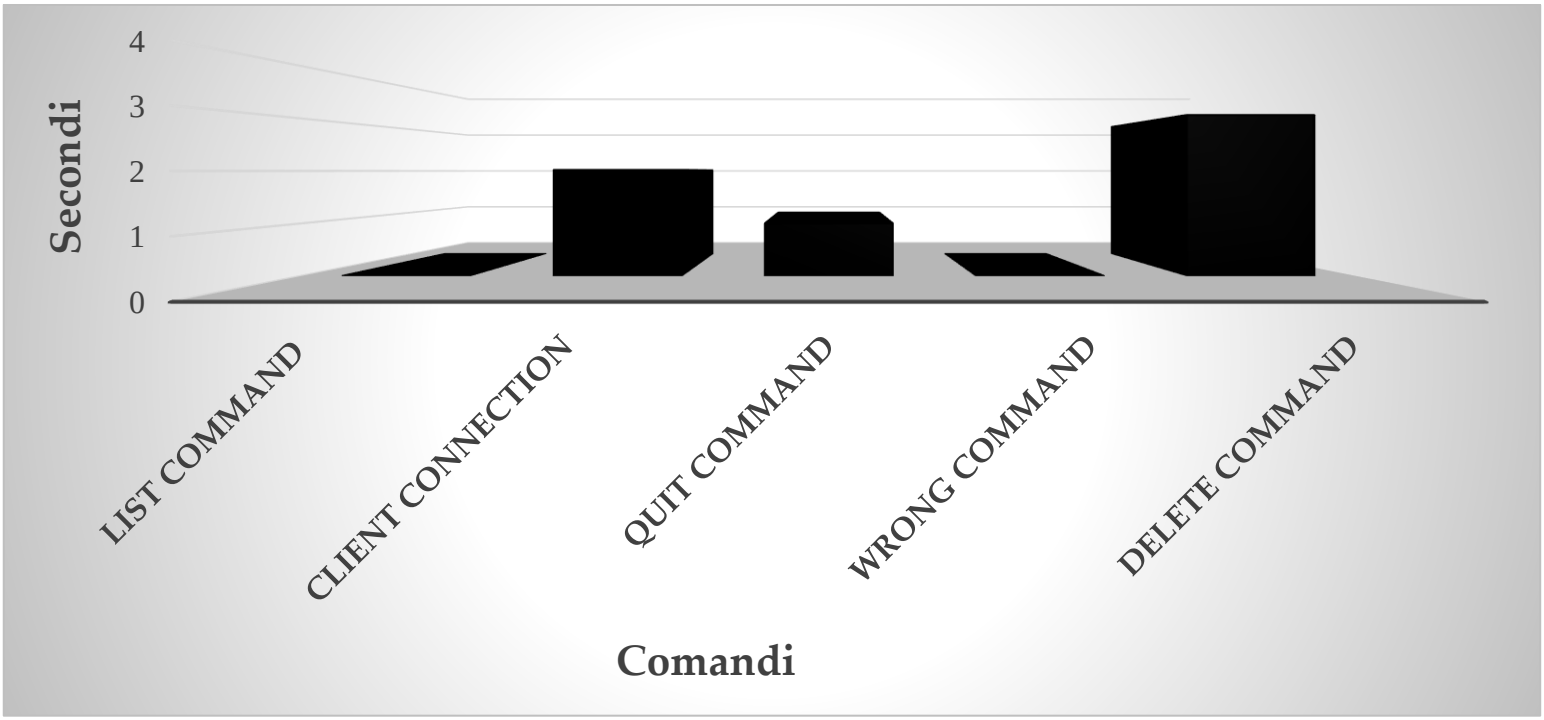
12.12. Tabella 12 – *put* File: N=50; WR/WS=25; T=10 s; P (Probabilità di Perdita)

	Peso File	P=20%
BancaItalia.txt	62.3 KB	0.033862 s
file–sample1.doc	100.4 KB	0.033447 s
file–sample2.pdf	1.3 MB	0.319965 s
file–sample3.mp3	3.9 MB	1.129615 s
file–sample5.jpeg	5.6 MB	1.631401 s
20mb.zip	20 MB	5.888419 s
file–sample4.mp4	38.2 MB	10.630280 s
100mb–fake.zip	104.7 MB	28.594000 s



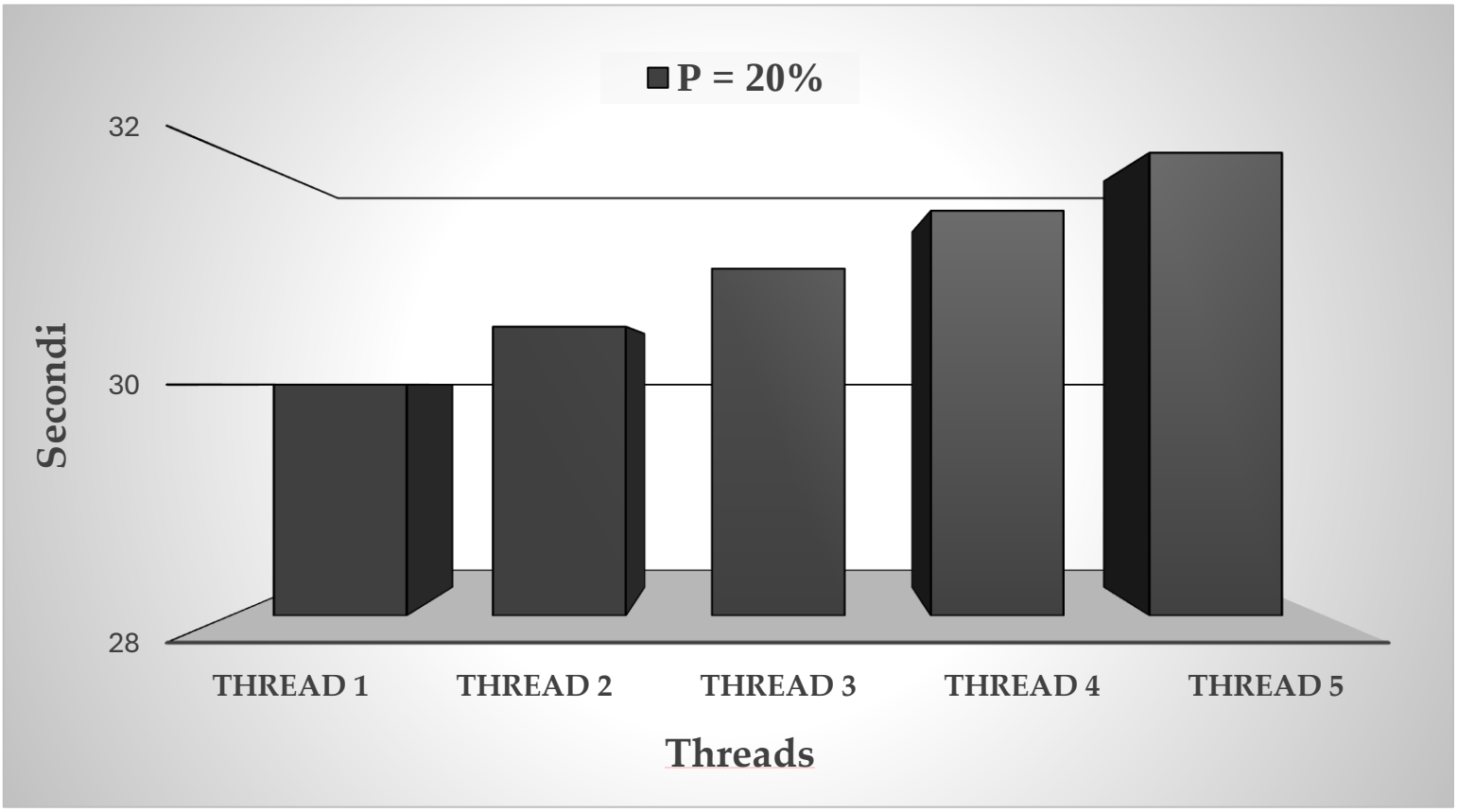
12.13. Tabella 13

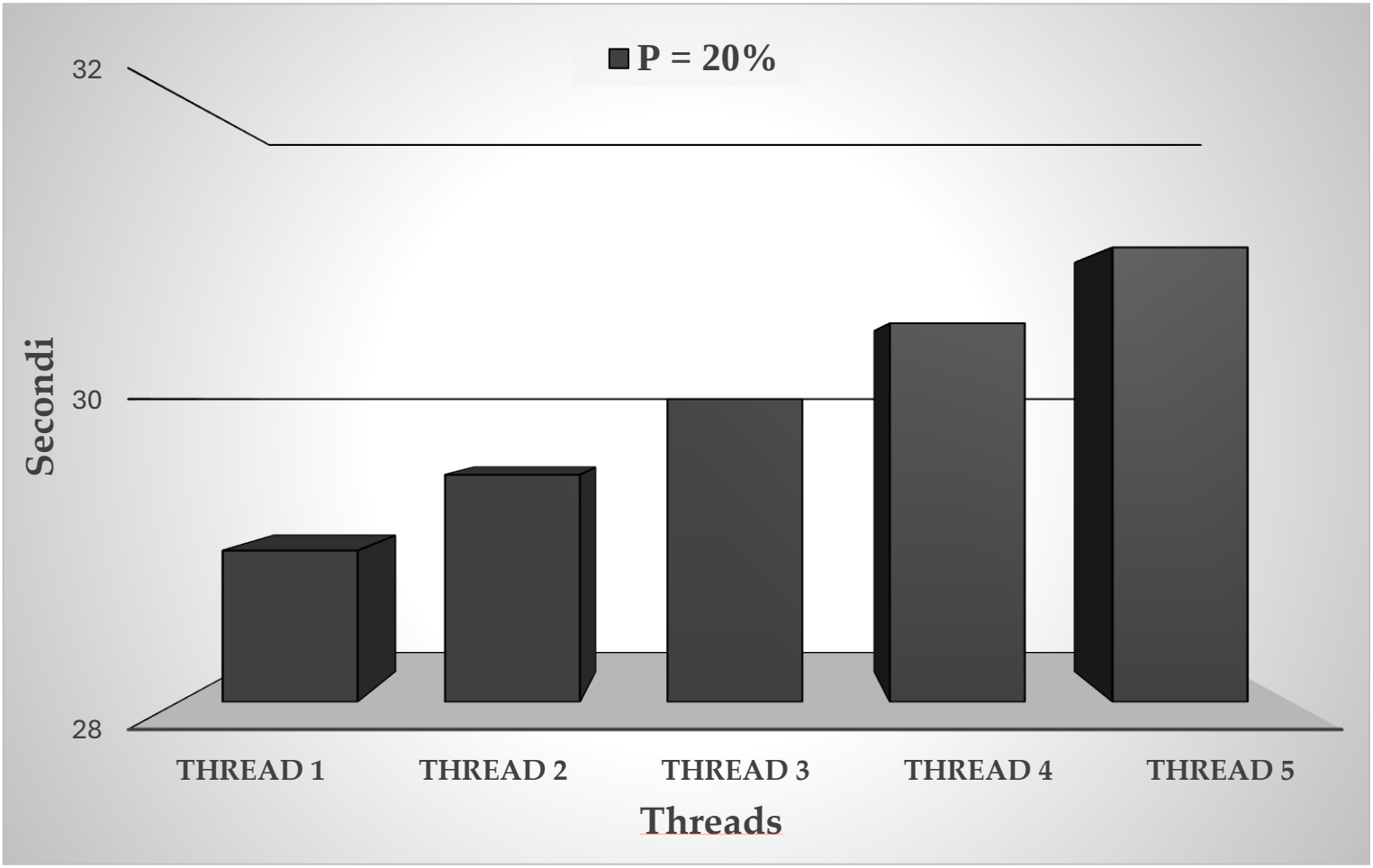
***list* Command**	0.001392 s
Client Connection	2.021213 s
***quit* Command**	1.000557 s
Wrong Command	0.000170 s
***delete* Command**	3.065422 s



12.14. Tabella 14 – *get* 100mb–fake.zip: N=50; WR/WS=25; T adattivo; P (Probabilità di Perdita)

	P=20%
Thread 1	30 s
Thread 2	30.5 s
Thread 3	31 s
Thread 4	31.5 s
Thread 5	32 s



12.15. Tabella 15 – *put* 100mb–fake.zip: N=50; WR/WS=25; T adattivo; P (Probabilità di Perdita)

	P=20%
Thread 1	29 s
Thread 2	29.5 s
Thread 3	30 s
Thread 4	30.5 s
Thread 5	31 s